



Introduction to Computer Networks (SWE3022)

Jaehoon (Paul) Jeong

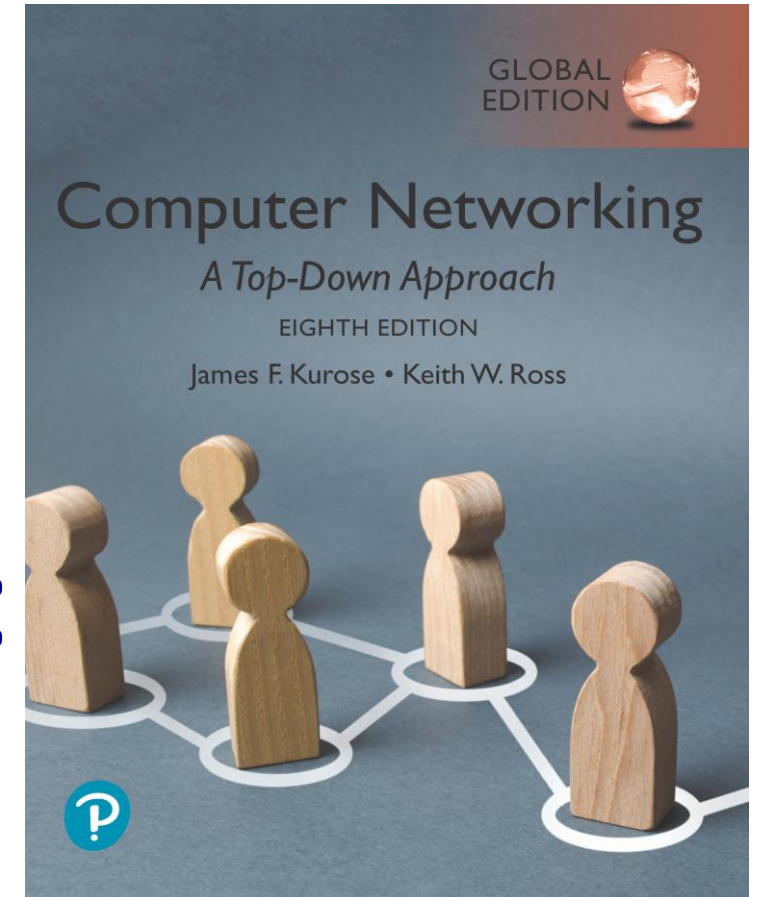
Department of Computer Science and Engineering

Sungkyunkwan University

Email: pauljeong@skku.edu

Note: The slides are adapted from the slides of Pearson Education Ltd.

Chapter 5. Network Layer - Control Plane: Part 2



Computer Networking: A Top-Down Approach

8th Edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

Network layer: “control plane” roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- **routing among ISPs: BGP**
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
 - SNMP
 - NETCONF/YANG

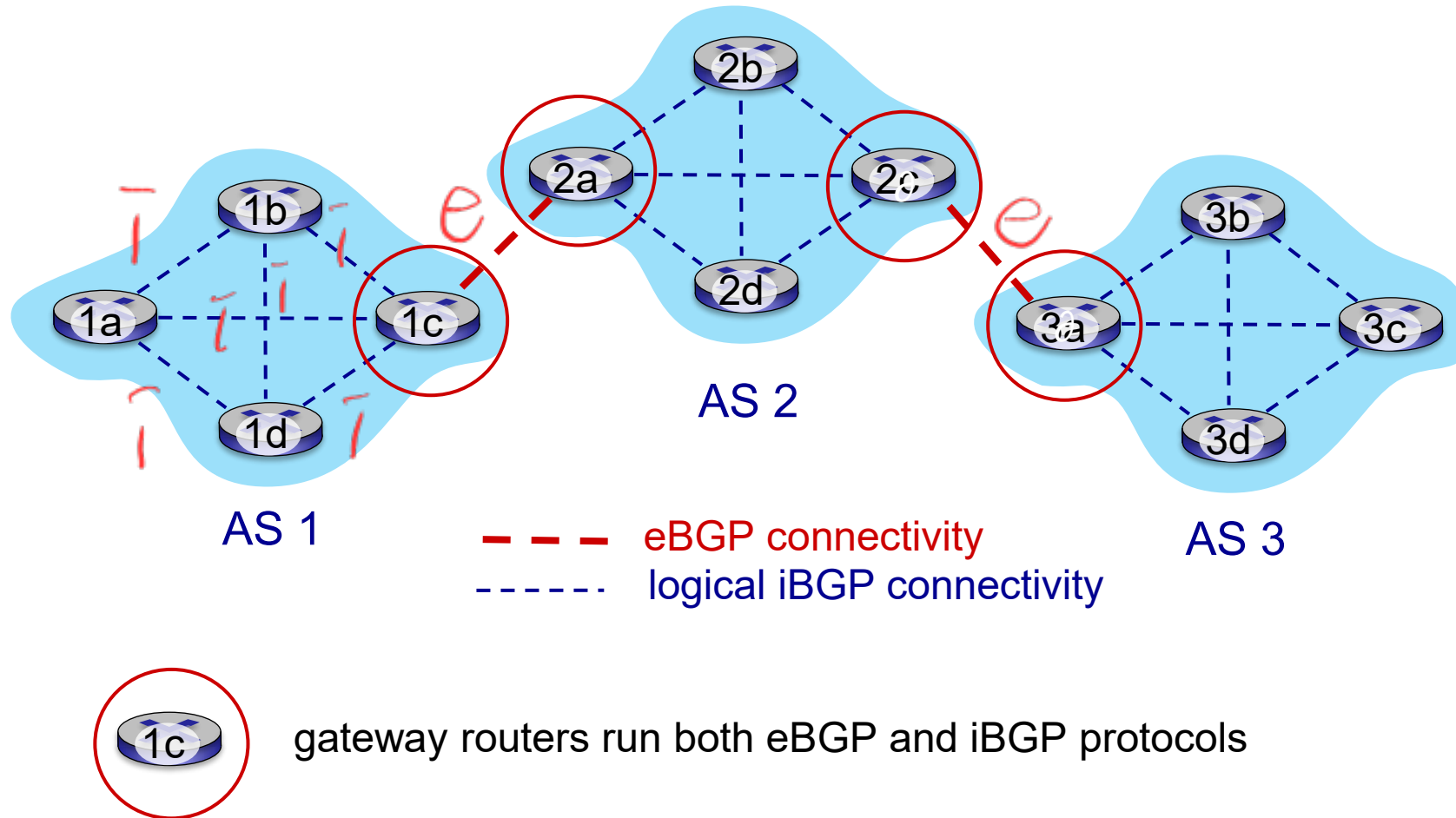
Internet inter-AS routing: BGP

- **BGP (Border Gateway Protocol):** *the de facto inter-domain routing protocol*
 - “glue that holds the Internet together”
- allows subnet to advertise its existence, and the destinations it can reach, to rest of Internet: *“I am here, here is who I can reach, and how”*
- BGP provides each AS a means to:
 - **eBGP:** obtain subnet reachability information from neighboring ASes
 - **iBGP:** propagate reachability information to all AS-internal routers.
 - determine “good” routes to other networks based on reachability information and *policy*

아웃 246번을
기대함

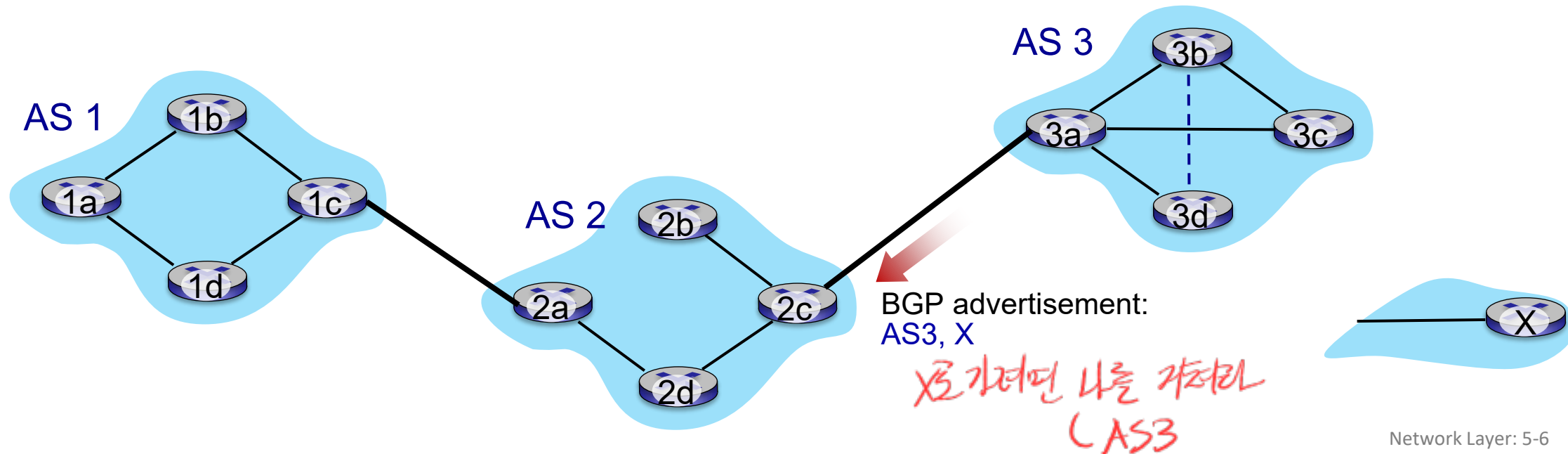
AS 내부
246번에 246번 246번

eBGP, iBGP connections



BGP basics

- **BGP session:** two BGP routers (“peers”) exchange BGP messages over semi-permanent TCP connection:
 - advertising *paths* to different destination network prefixes (BGP is a “path vector” protocol)
- when AS3 gateway 3a advertises *path AS3, X* to AS2 gateway 2c:
 - AS3 *promises* to AS2 it will forward datagrams towards X



Path attributes and BGP routes

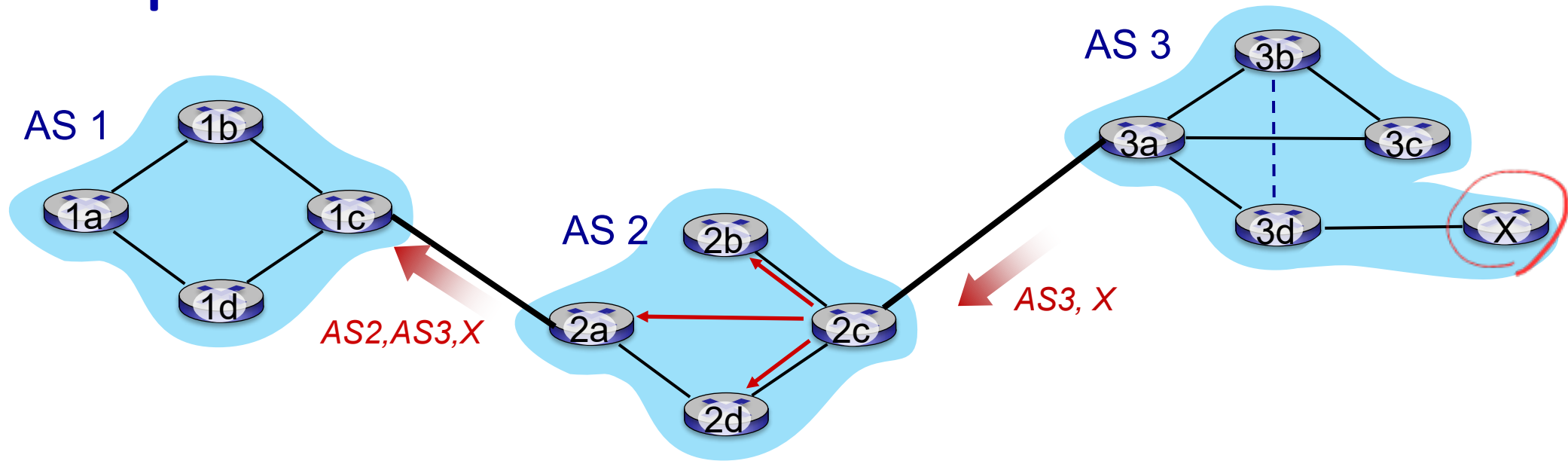
- BGP advertised route: prefix + attributes

- prefix: destination being advertised
- two important attributes:
 - **AS-PATH**: list of ASes through which prefix advertisement has passed
 - **NEXT-HOP**: indicates specific internal-AS router to next-hop AS

- **policy-based routing**: BGP는 policy 기반

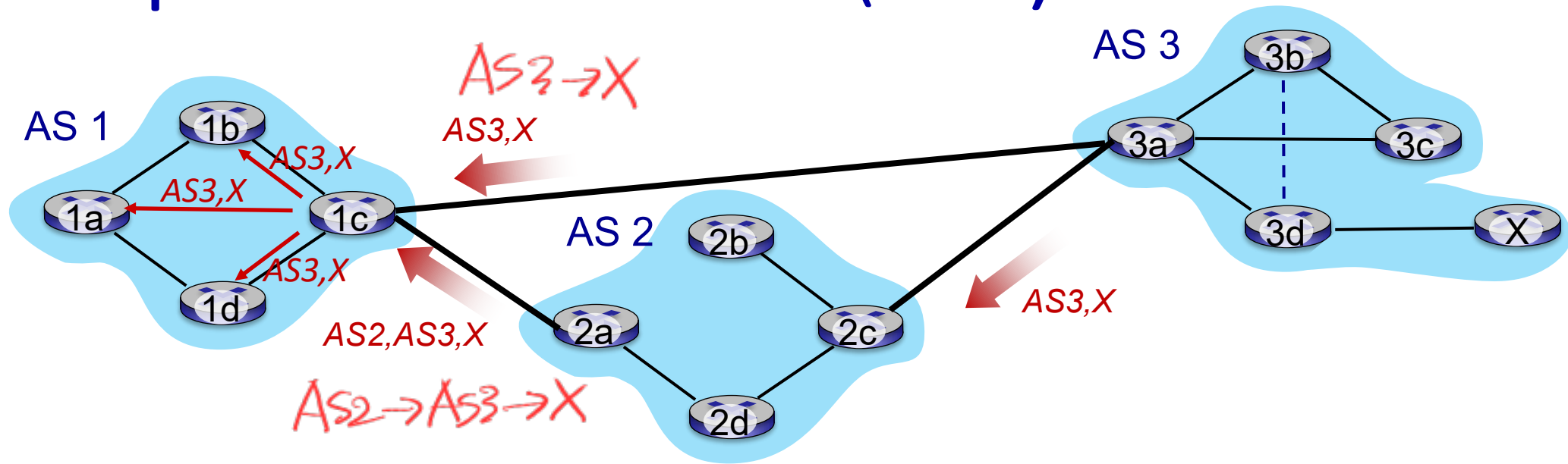
- gateway receiving route advertisement uses *import policy* to accept/decline path (e.g., never route through AS Y).
- AS policy also determines whether to *advertise* path to other neighboring ASes

BGP path advertisement



- AS2 router 2c receives path advertisement **AS3, X** (via eBGP) from AS3 router 3a
- based on AS2 policy, AS2 router 2c accepts path AS3, X, propagates (via iBGP) to all AS2 routers
- based on AS2 policy, AS2 router 2a advertises (via eBGP) path **AS2, AS3, X** to AS1 router 1c

BGP path advertisement (more)



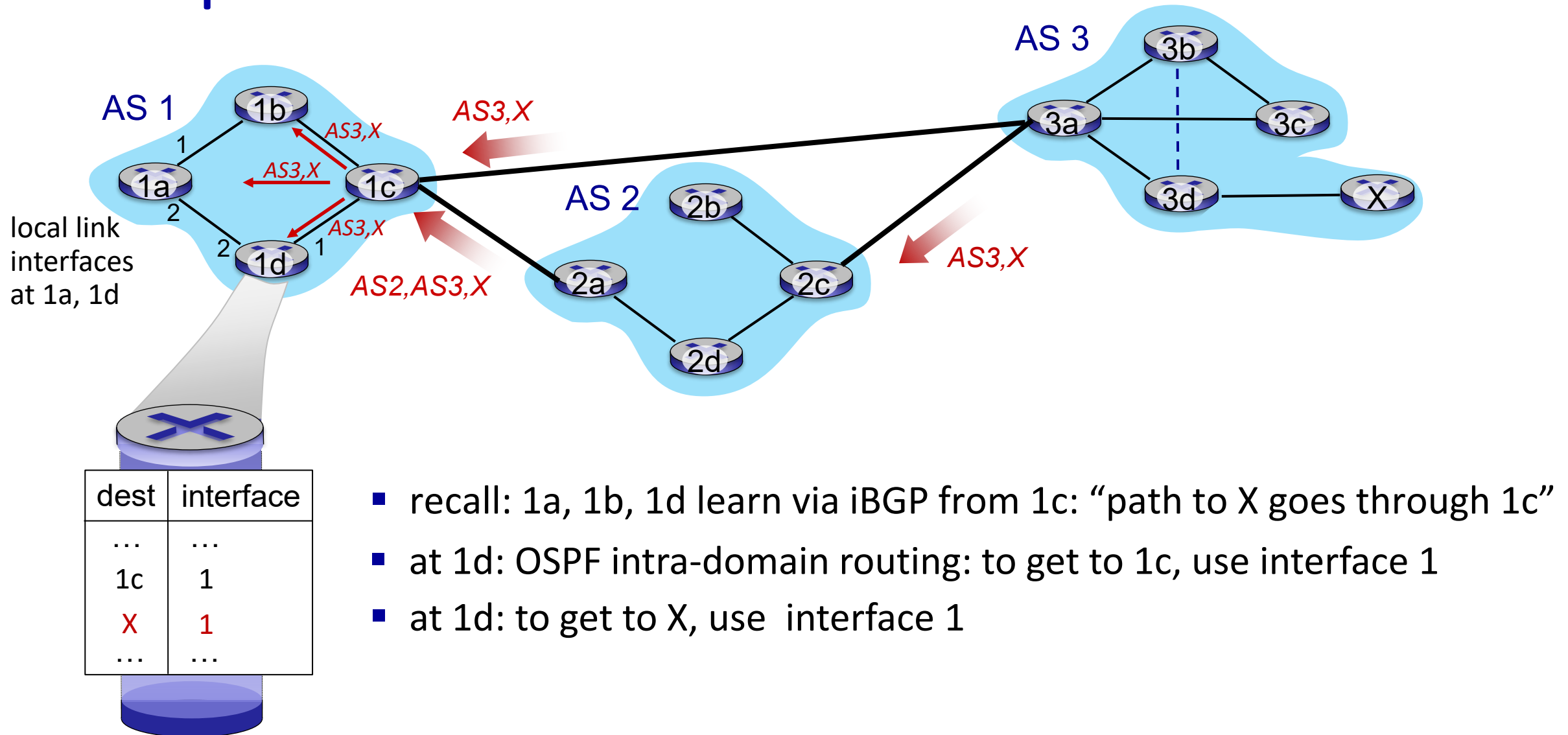
gateway router may learn **multiple** paths to destination:

- AS1 gateway router 1c learns path **AS2, AS3, X** from 2a
- AS1 gateway router 1c learns path **AS3, X** from 3a
- based on **policy**, AS1 gateway router 1c chooses path **AS3, X** and advertises path within AS1 via iBGP

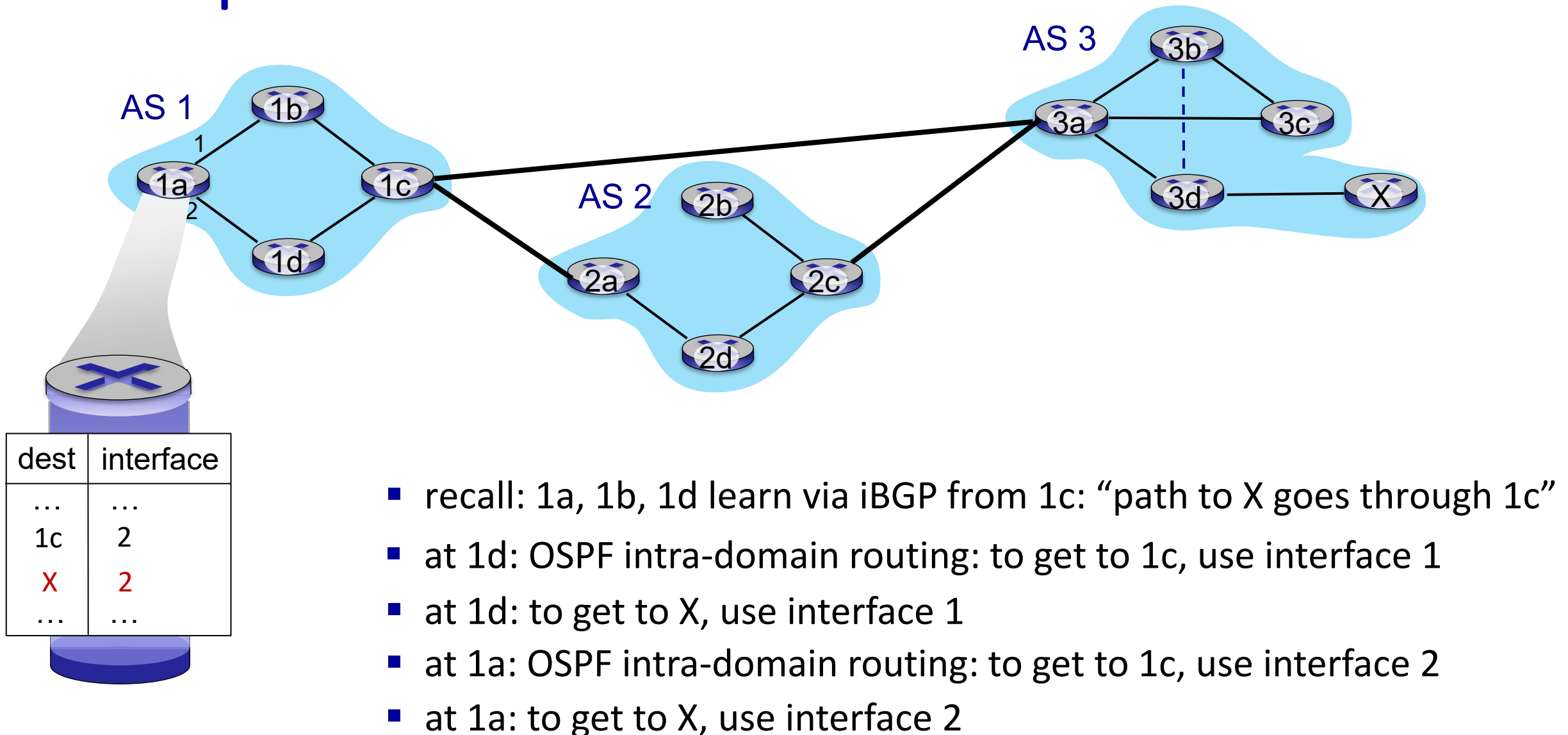
BGP messages

- BGP messages exchanged between peers over TCP connection
- BGP messages:
 - **OPEN**: opens TCP connection to remote BGP peer and authenticates sending BGP peer
 - **UPDATE**: advertises new path (or withdraws old)
 - **KEEPALIVE**: keeps connection alive in absence of UPDATES; also ACKs OPEN request
 - **NOTIFICATION**: reports errors in previous msg; also used to close connection

BGP path advertisement



BGP path advertisement



Why different Intra-, Inter-AS routing ?

policy:

- inter-AS: admin wants control over how its traffic is routed, who routes through its network
- intra-AS: single admin, so policy is less of an issue

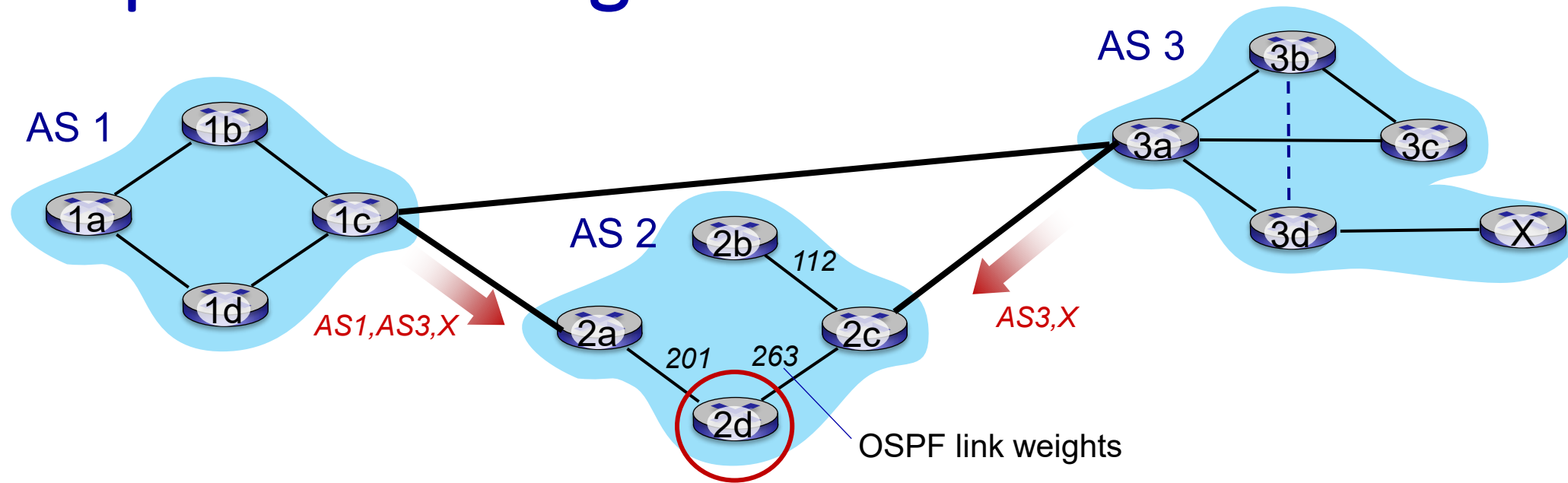
scale:

- hierarchical routing saves table size, reduced update traffic

performance:

- intra-AS: can focus on performance
- inter-AS: policy dominates over performance

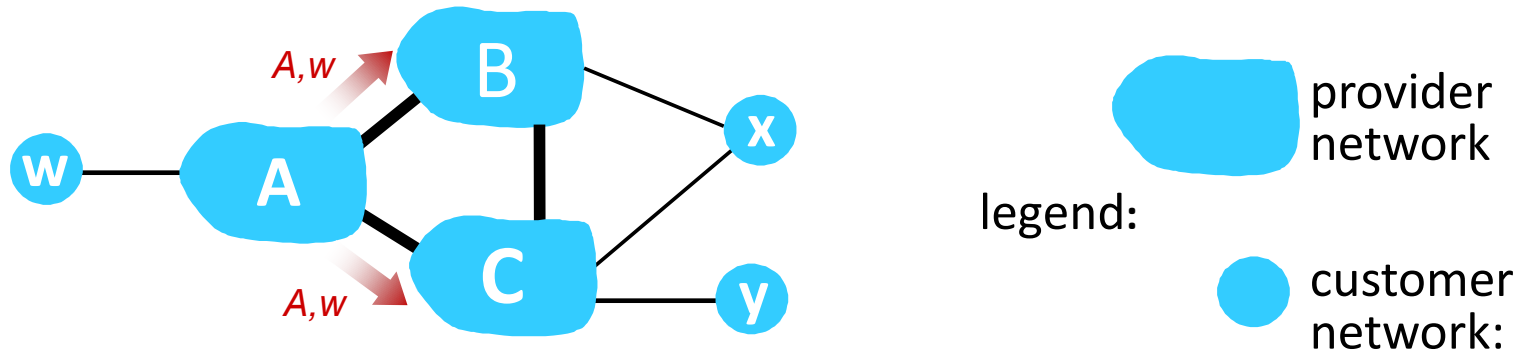
Hot potato routing



cost가 2a < 2c니까 2a 선택 (외부 비용 생각X)

- 2d learns (via iBGP) it can route to X via 2a or 2c
- **hot potato routing**: choose local gateway that has least *intra-domain* cost (e.g., 2d chooses 2a, even though more AS hops to X): don't worry about inter-domain cost!

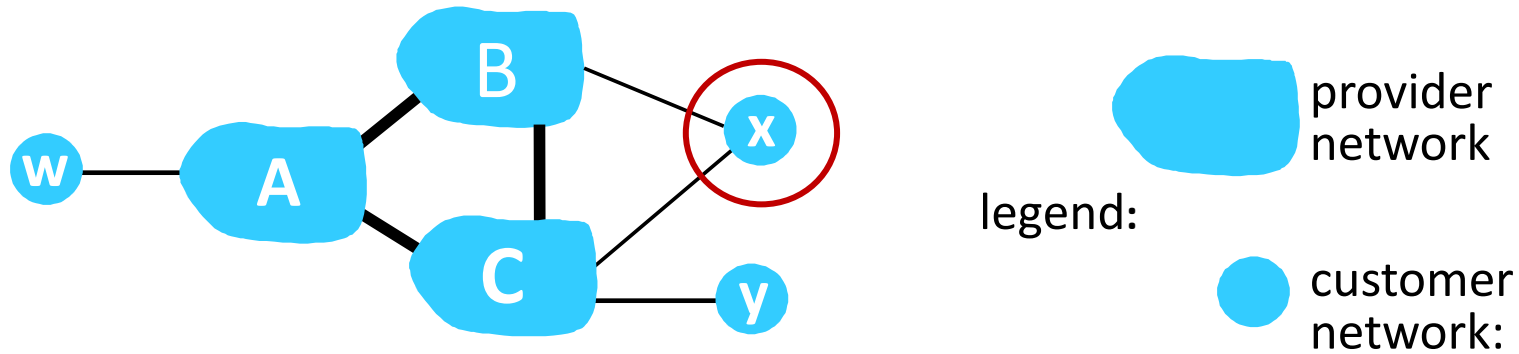
BGP: achieving policy via advertisements



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical “real world” policy)

- A advertises path Aw to B and to C
- B *chooses not to advertise* BA_w to C!
 - B gets no “revenue” for routing CBA_w, since none of C, A, w are B’s customers
 - C does *not* learn CBA_w path
- C will route CA_w (not using B) to get to w

BGP: achieving policy via advertisements (more)



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical “real world” policy)

- A,B,C are **provider networks**
- x,w,y are **customer** (of provider networks)
- x is **dual-homed**: attached to two networks
- **policy to enforce**: x does not want to route from B to C via x
 - .. so x will not advertise to B a route to C

BGP route selection

- router may learn more than one route to destination AS, selects route based on:
 1. local preference value attribute: policy decision
 2. shortest AS-PATH
 3. closest NEXT-HOP router: hot potato routing
 4. additional criteria

Network layer: “control plane” roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- **SDN control plane**
- Internet Control Message Protocol



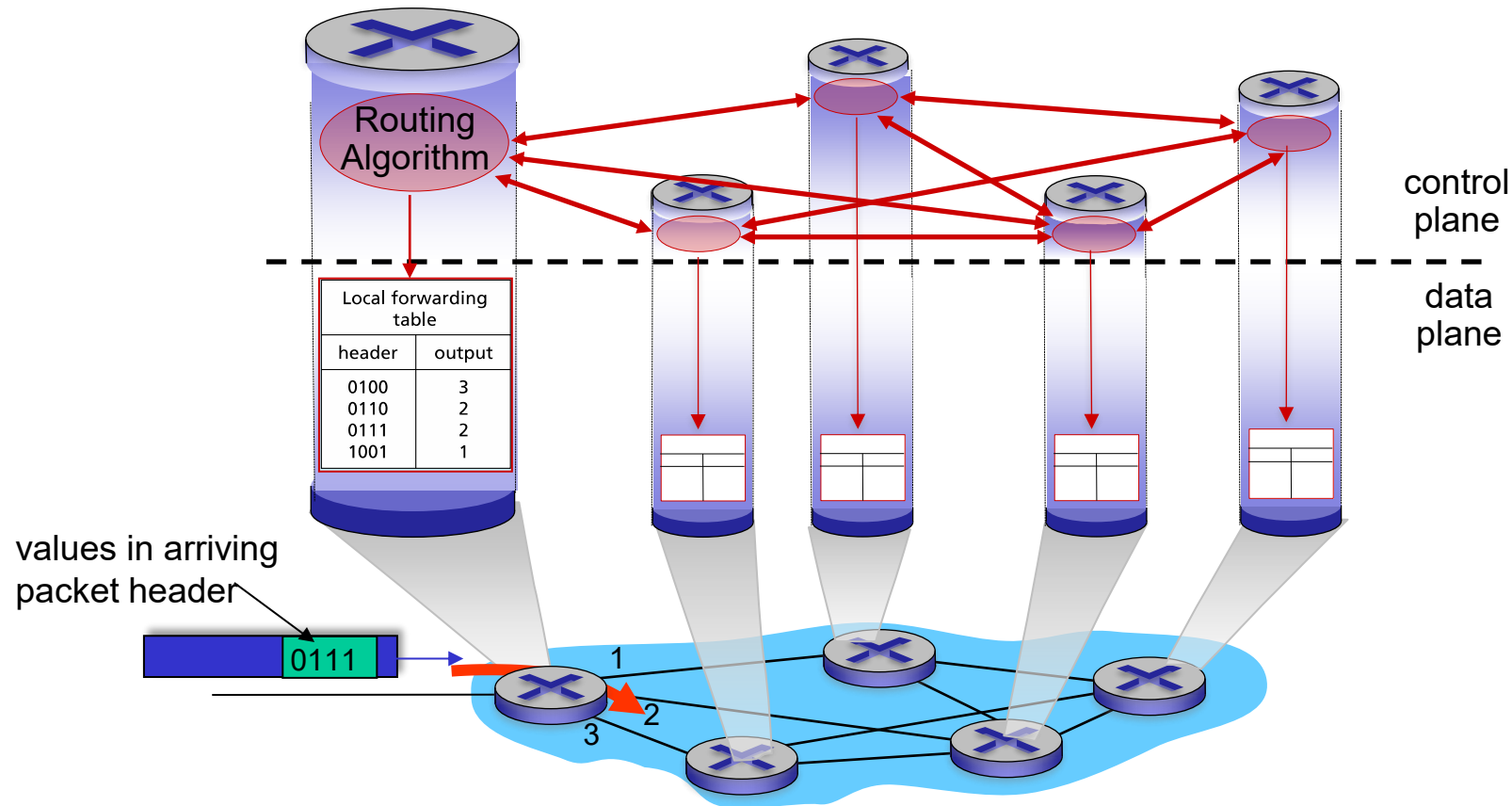
- network management, configuration
 - SNMP
 - NETCONF/YANG

Software-Defined Networking (SDN)

- Internet network layer: historically implemented via distributed, per-router control approach:
 - *monolithic* router contains switching hardware, runs proprietary implementation of Internet standard protocols (IP, RIP, IS-IS, OSPF, BGP) in proprietary router OS (e.g., Cisco IOS)
 - different “middleboxes” for different network layer functions: firewalls, load balancers, NAT boxes, ..
- ~2005: renewed interest in rethinking network control plane

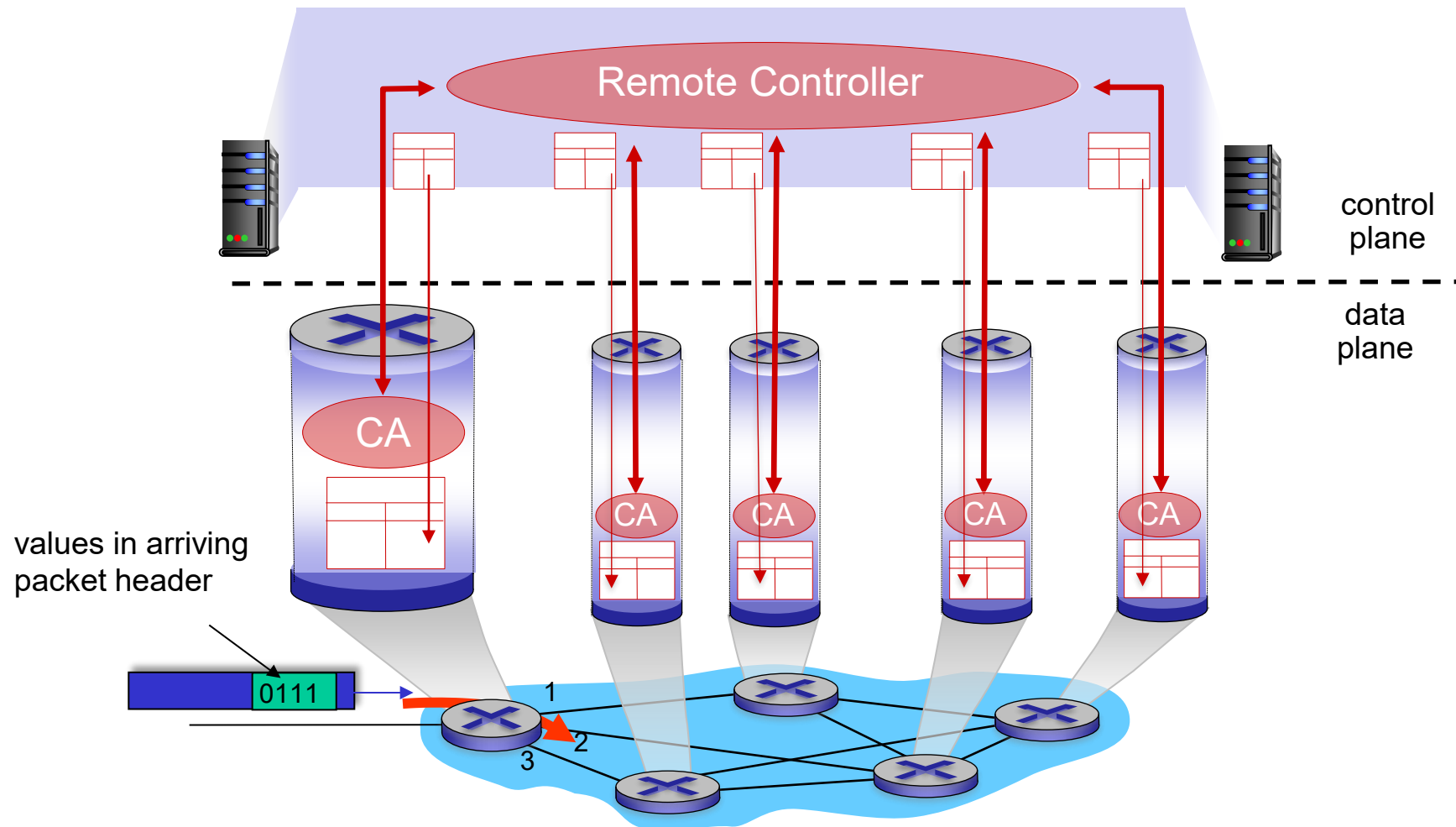
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane to compute forwarding tables



Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers

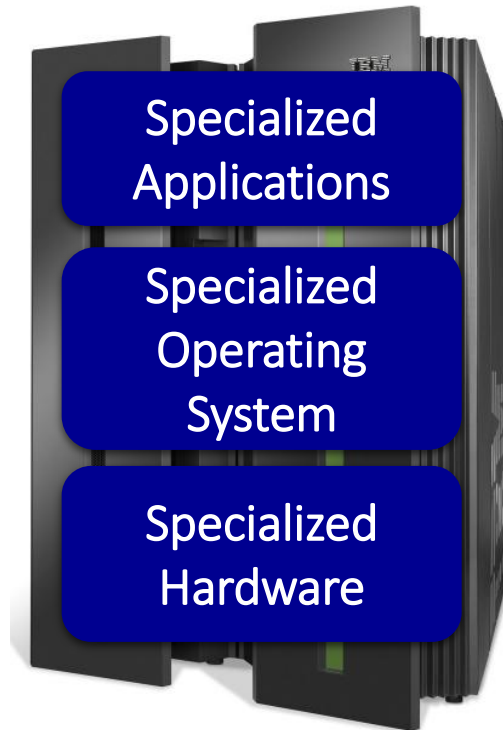


Software-Defined Networking (SDN)

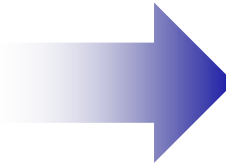
Why a *logically centralized* control plane?

- easier network management: avoid router misconfigurations, greater flexibility of traffic flows
- table-based forwarding (recall OpenFlow API) allows “programming” routers
 - centralized “programming” easier: compute tables centrally and distribute
 - distributed “programming” more difficult: compute tables as result of distributed algorithm (protocol) implemented in each-and-every router
- open (non-proprietary) implementation of control plane
 - foster innovation: let 1000 flowers bloom

SDN analogy: mainframe to PC revolution



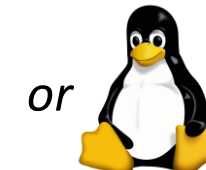
Vertically integrated
Closed, proprietary
Slow innovation
Small industry



— Open Interface —



Windows



or

Linux



or

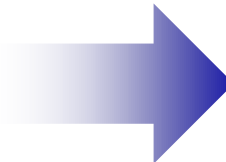
MAC OS

— Open Interface —

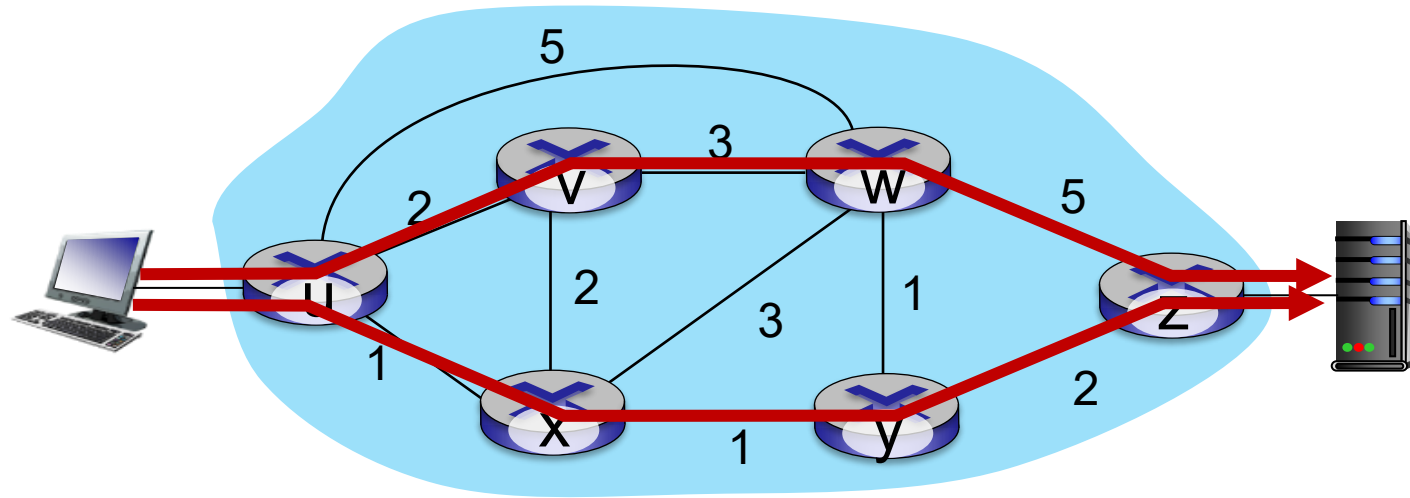


Microprocessor

Horizontal
Open interfaces
Rapid innovation
Huge industry



Traffic engineering: difficult with traditional routing

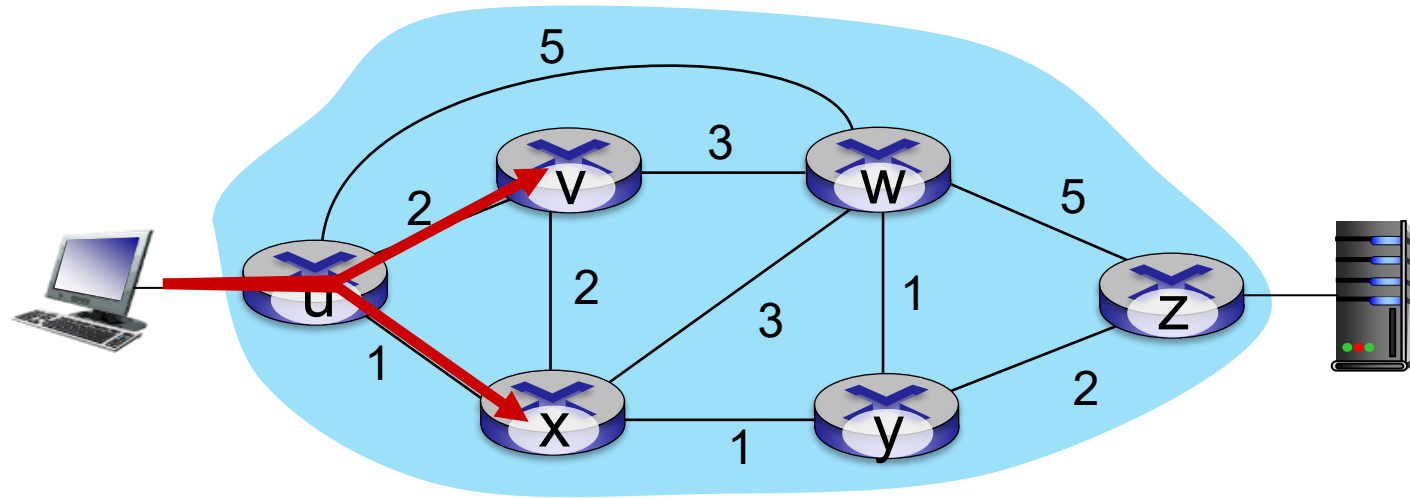


Q: what if network operator wants u-to-z traffic to flow along *uvwz*, rather than *uxyz*?

A: need to re-define link weights so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

link weights are only control “knobs”: not much control!

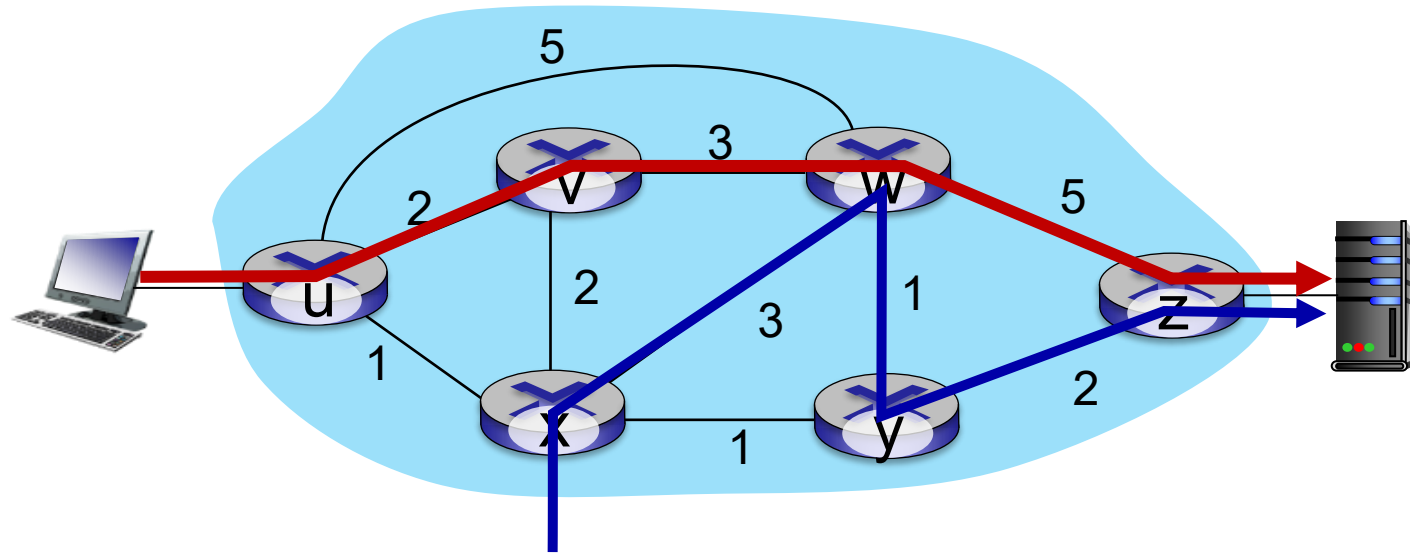
Traffic engineering: difficult with traditional routing



Q: what if network operator wants to split u-to-z traffic along uvwz *and* uxyz (load balancing)?

A: can't do it (or need a new routing algorithm)

Traffic engineering: difficult with traditional routing

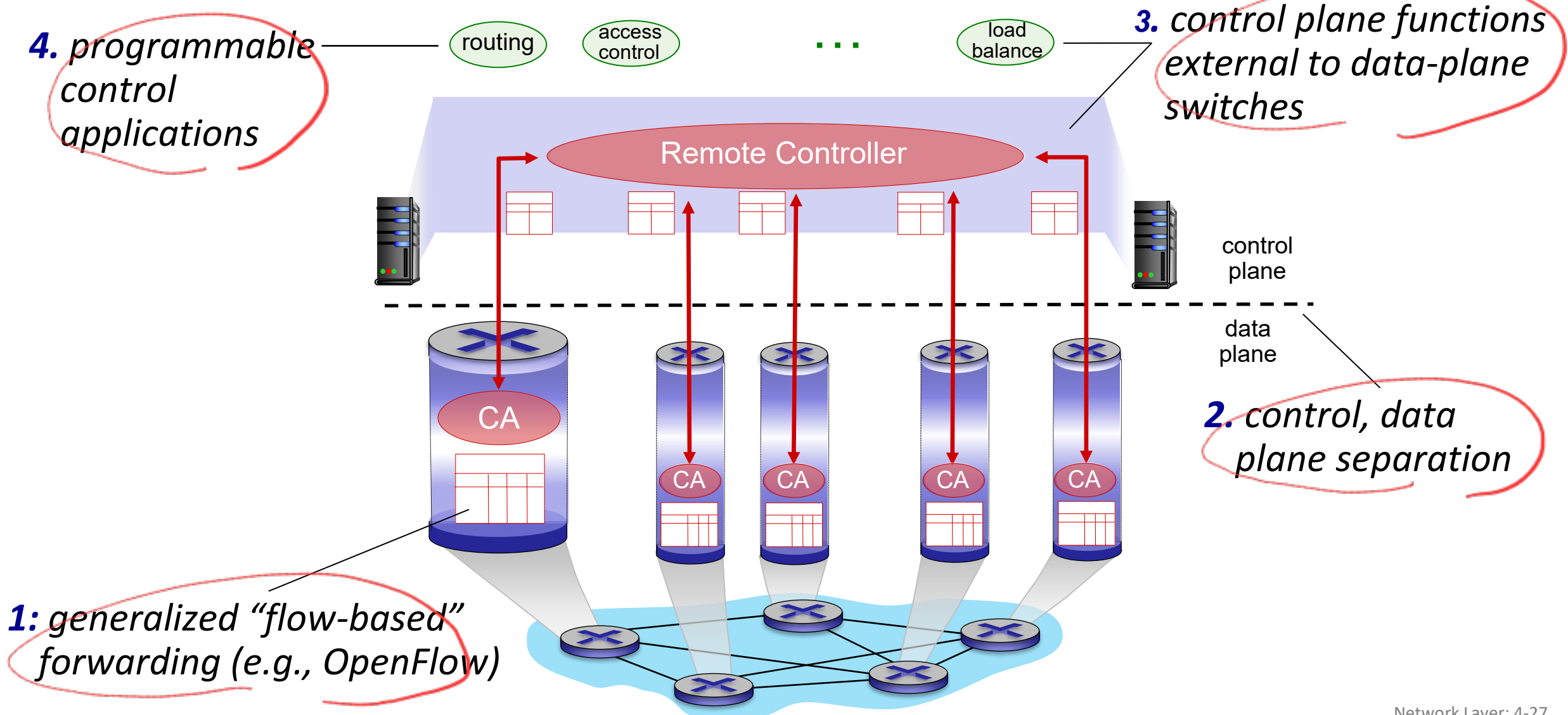


Q: what if w wants to route blue and red traffic differently from w to z?

A: can't do it (with destination-based forwarding, and LS, DV routing)

We learned in Chapter 4 that generalized forwarding and SDN can be used to achieve *any* routing desired

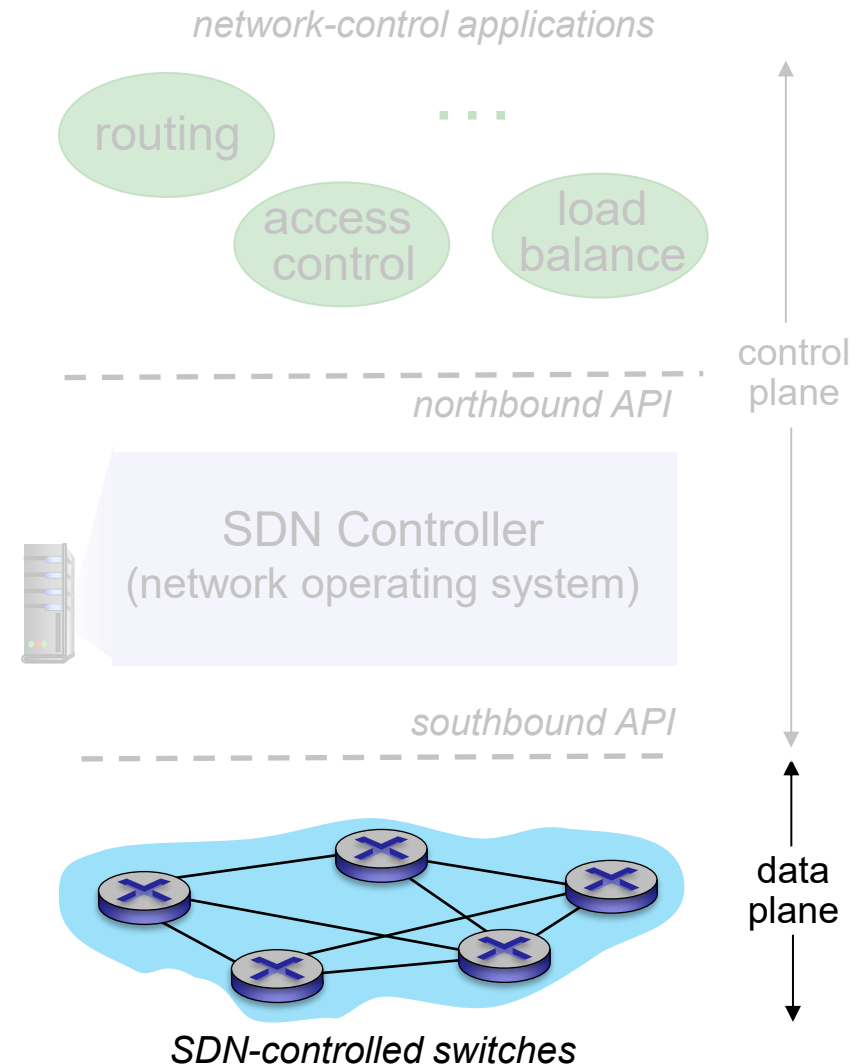
Software-Defined Networking (SDN)



Software-Defined Networking (SDN)

Data-plane switches:

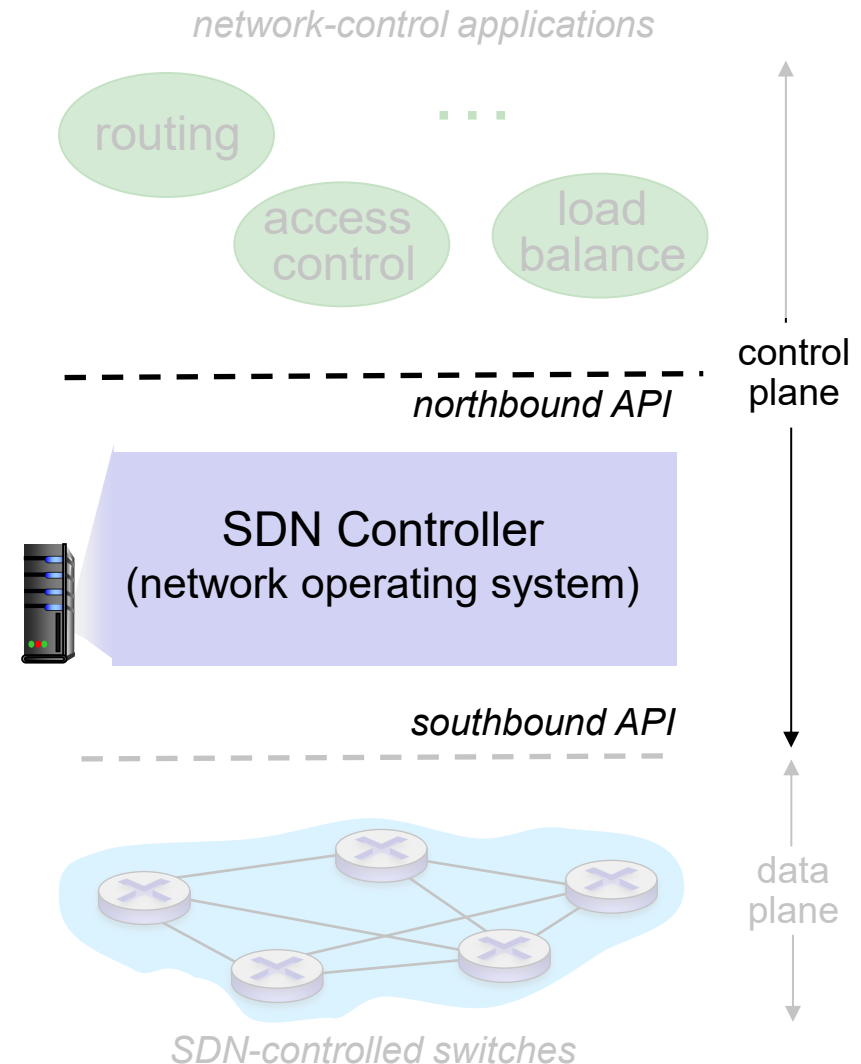
- fast, simple, commodity switches implementing generalized data-plane forwarding (Section 4.4) in hardware
- flow (forwarding) table computed, installed under controller supervision
- API for table-based switch control (e.g., OpenFlow)
 - defines what is controllable, what is not
- protocol for communicating with controller (e.g., OpenFlow)



Software-Defined Networking (SDN)

SDN controller (network OS):

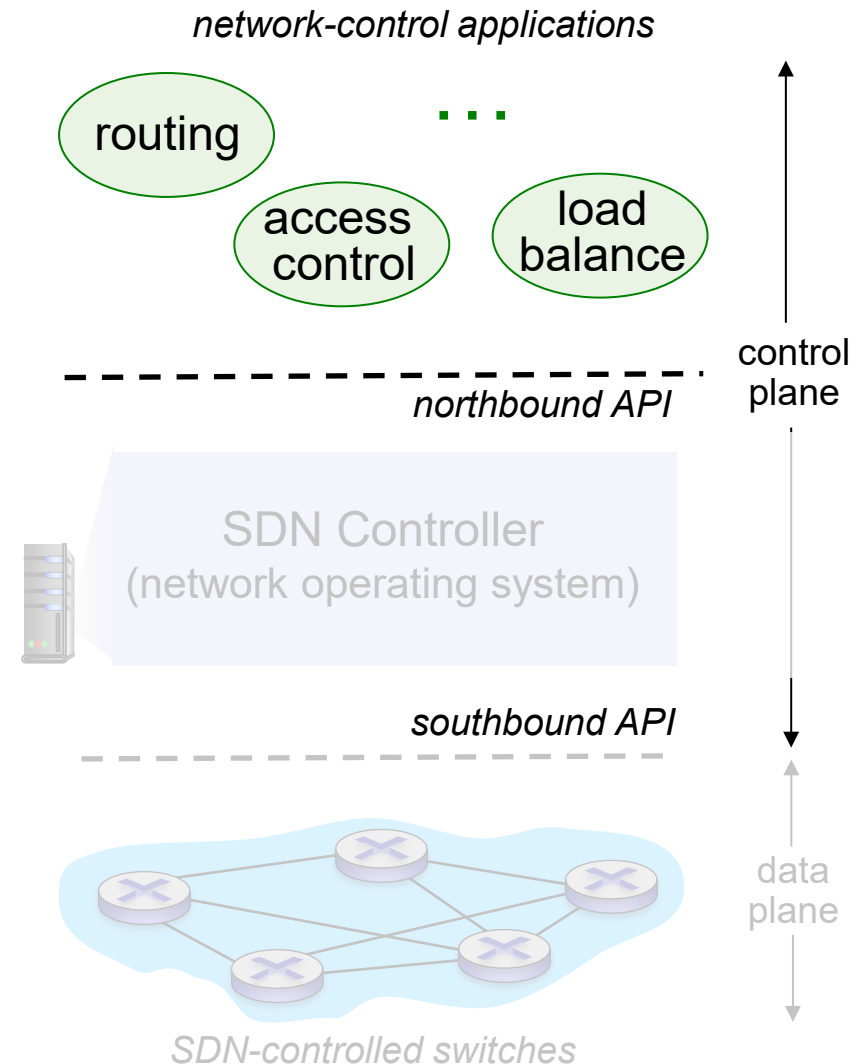
- maintain network state information
- interacts with network control applications “above” via northbound API
- interacts with network switches “below” via southbound API
- implemented as distributed system for performance, scalability, fault-tolerance, robustness



Software-Defined Networking (SDN)

network-control apps:

- “brains” of control: implement control functions using lower-level services, API provided by SDN controller
- *unbundled*: can be provided by 3rd party: distinct from routing vendor, or SDN controller

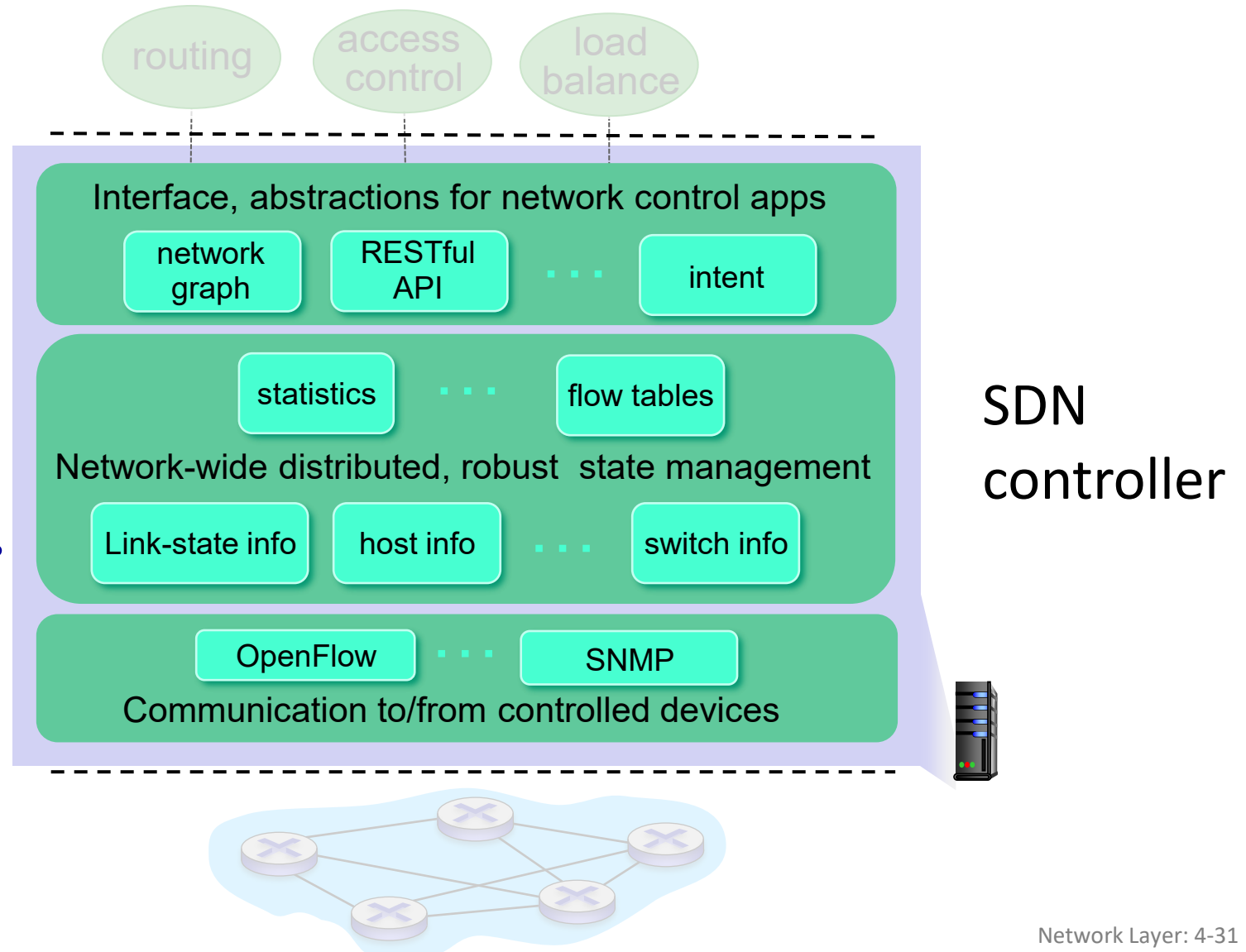


Components of SDN controller

interface layer to network
control apps: abstractions API

network-wide state
management : state of
networks links, switches,
services: a *distributed database*

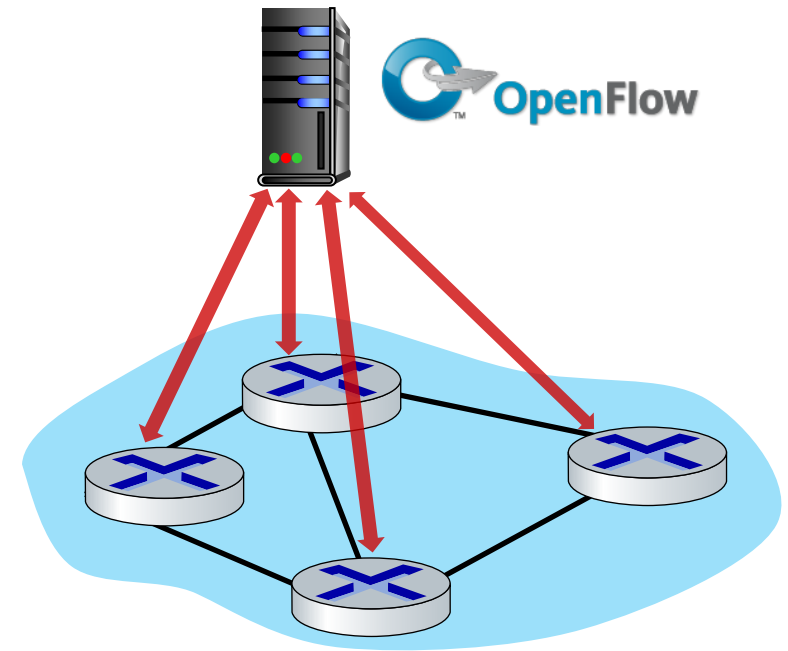
communication: communicate
between SDN controller and
controlled switches



OpenFlow protocol

- operates between controller, switch
- TCP used to exchange messages
 - optional encryption
- three classes of OpenFlow messages:
 - controller-to-switch
 - asynchronous (switch to controller)
 - symmetric (misc.)
- distinct from OpenFlow API
 - API used to specify generalized forwarding actions

OpenFlow Controller

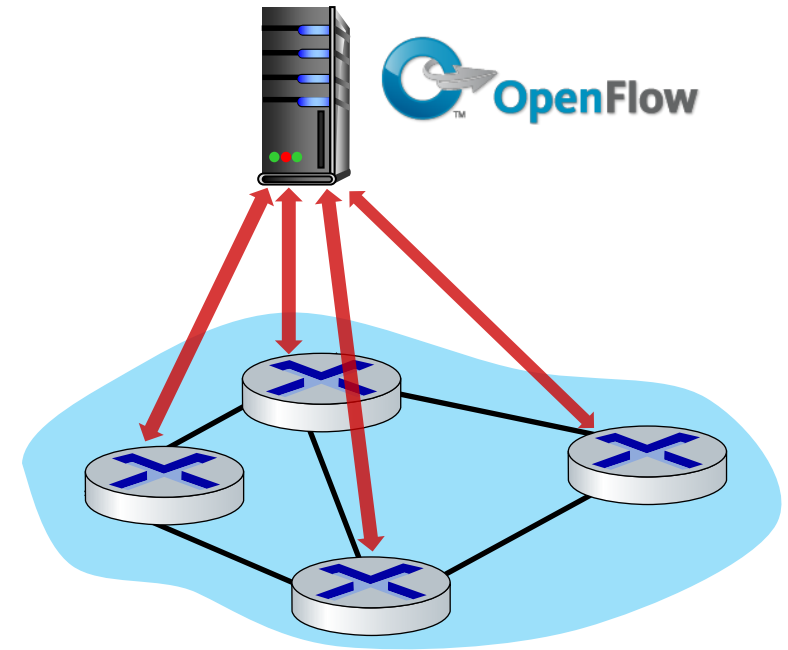


OpenFlow: controller-to-switch messages

Key controller-to-switch messages

- *features*: controller queries switch features, switch replies
- *configure*: controller queries/sets switch configuration parameters
- *modify-state*: add, delete, modify flow entries in the OpenFlow tables
- *packet-out*: controller can send this packet out of specific switch port

OpenFlow Controller

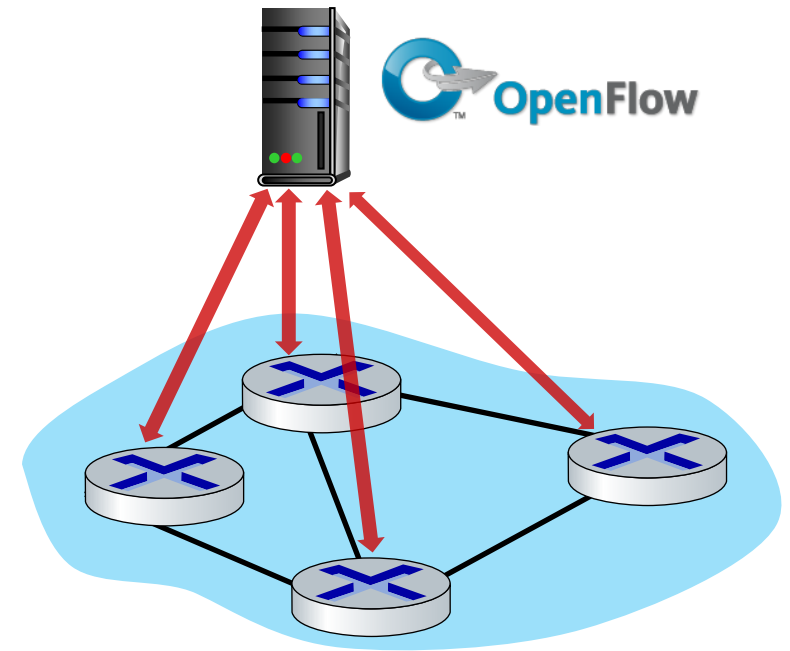


OpenFlow: switch-to-controller messages

Key switch-to-controller messages

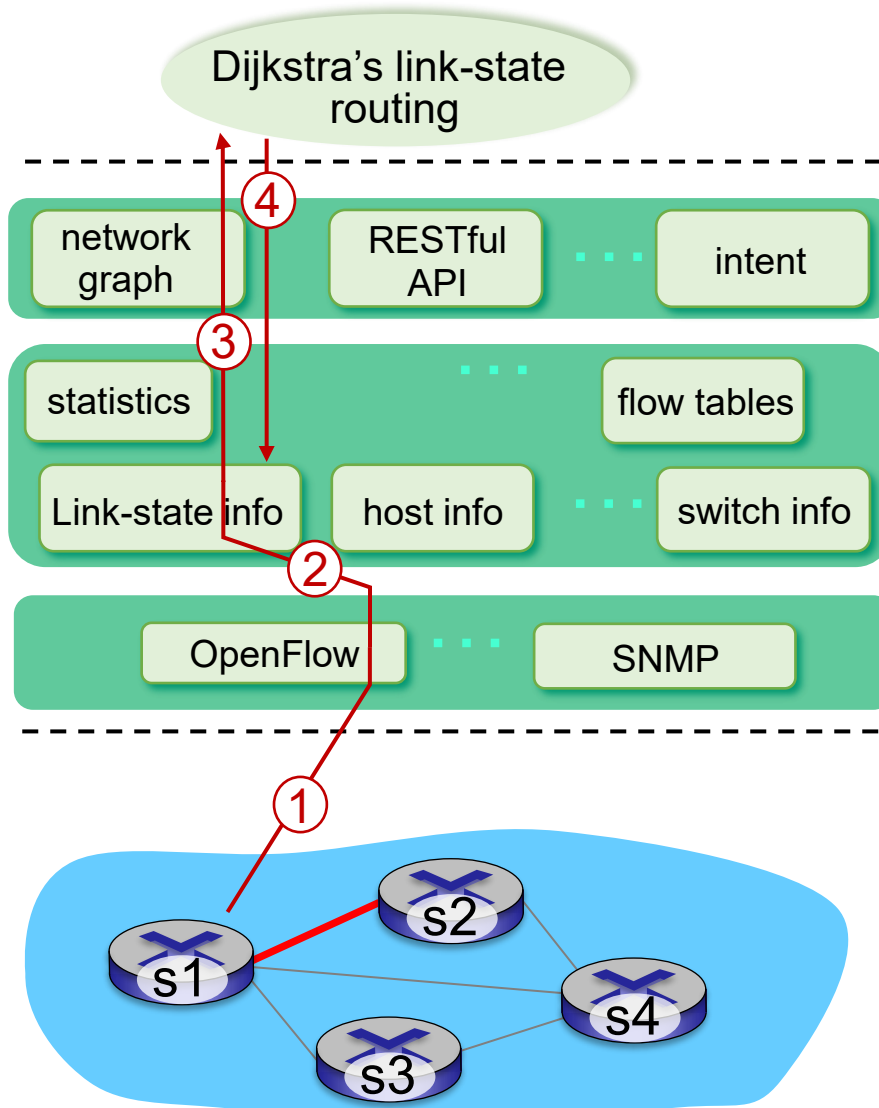
- *packet-in*: transfer packet (and its control) to controller. See packet-out message from controller
- *flow-removed*: flow table entry deleted at switch
- *port status*: inform controller of a change on a port.

OpenFlow Controller



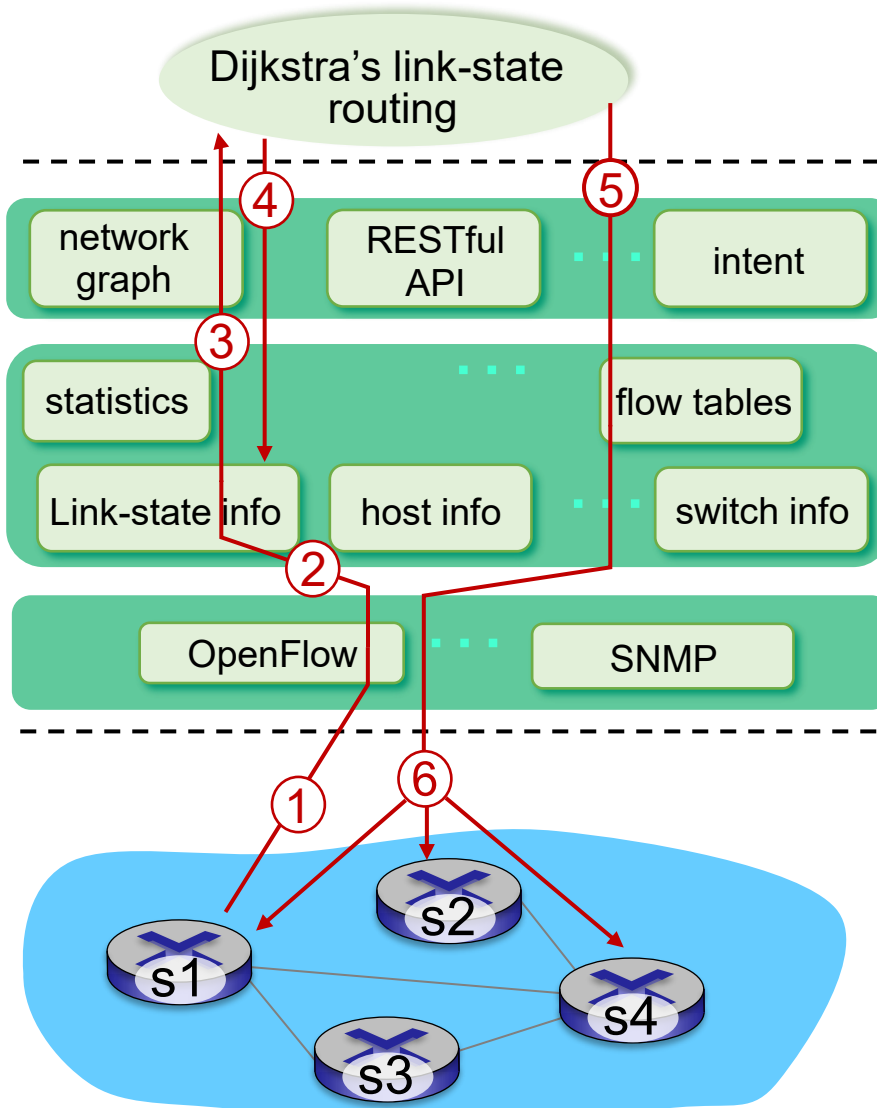
Fortunately, network operators don't "program" switches by creating/sending OpenFlow messages directly. Instead use higher-level abstraction at controller

SDN: control/data plane interaction example



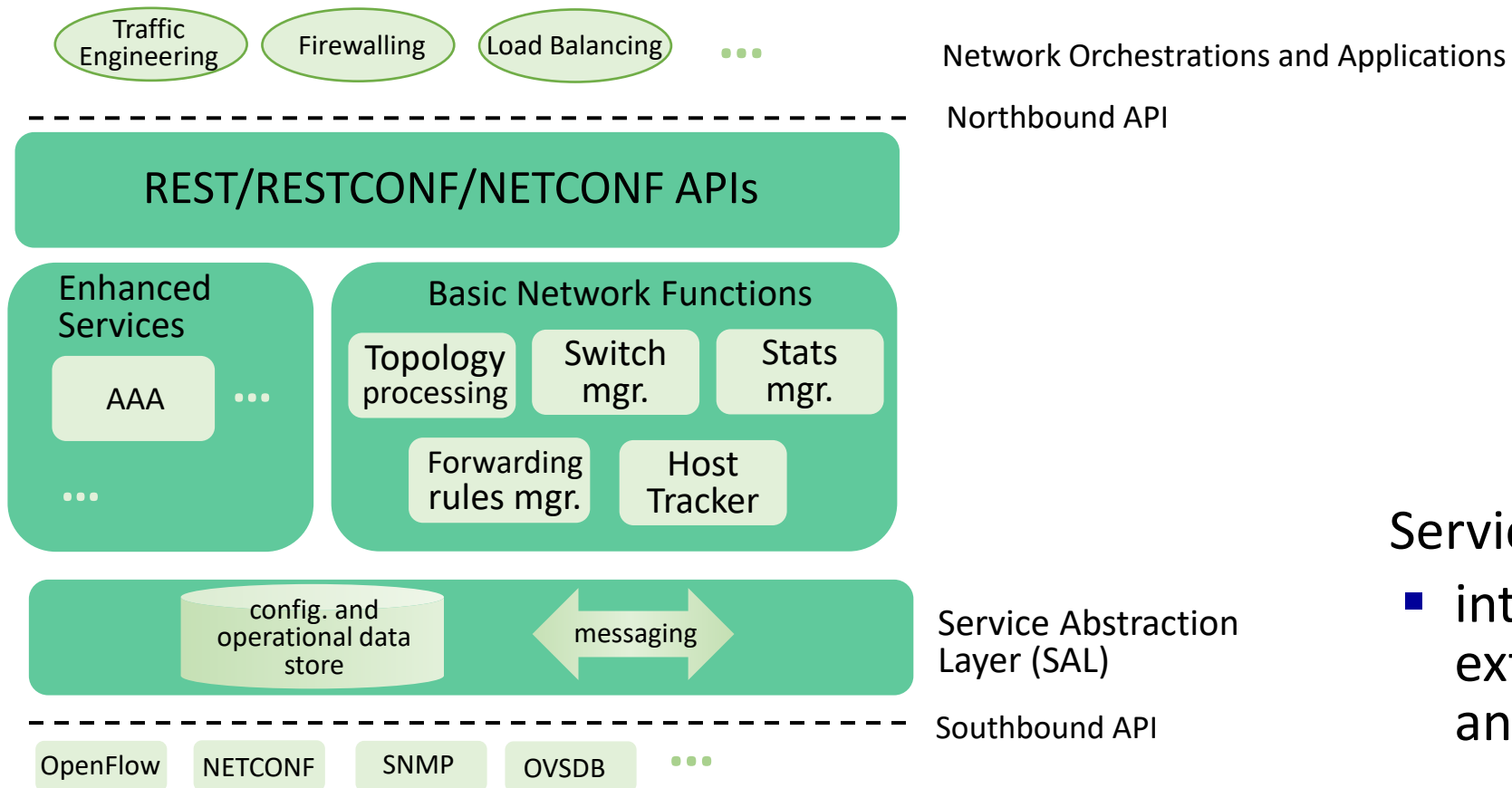
- ① S1, experiencing link failure uses OpenFlow port status message to notify controller
- ② SDN controller receives OpenFlow message, updates link status info
- ③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.
- ④ Dijkstra's routing algorithm accesses network graph info, link state info in controller, computes new routes

SDN: control/data plane interaction example



- ⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed
- ⑥ controller uses OpenFlow to install new tables in switches that need updating

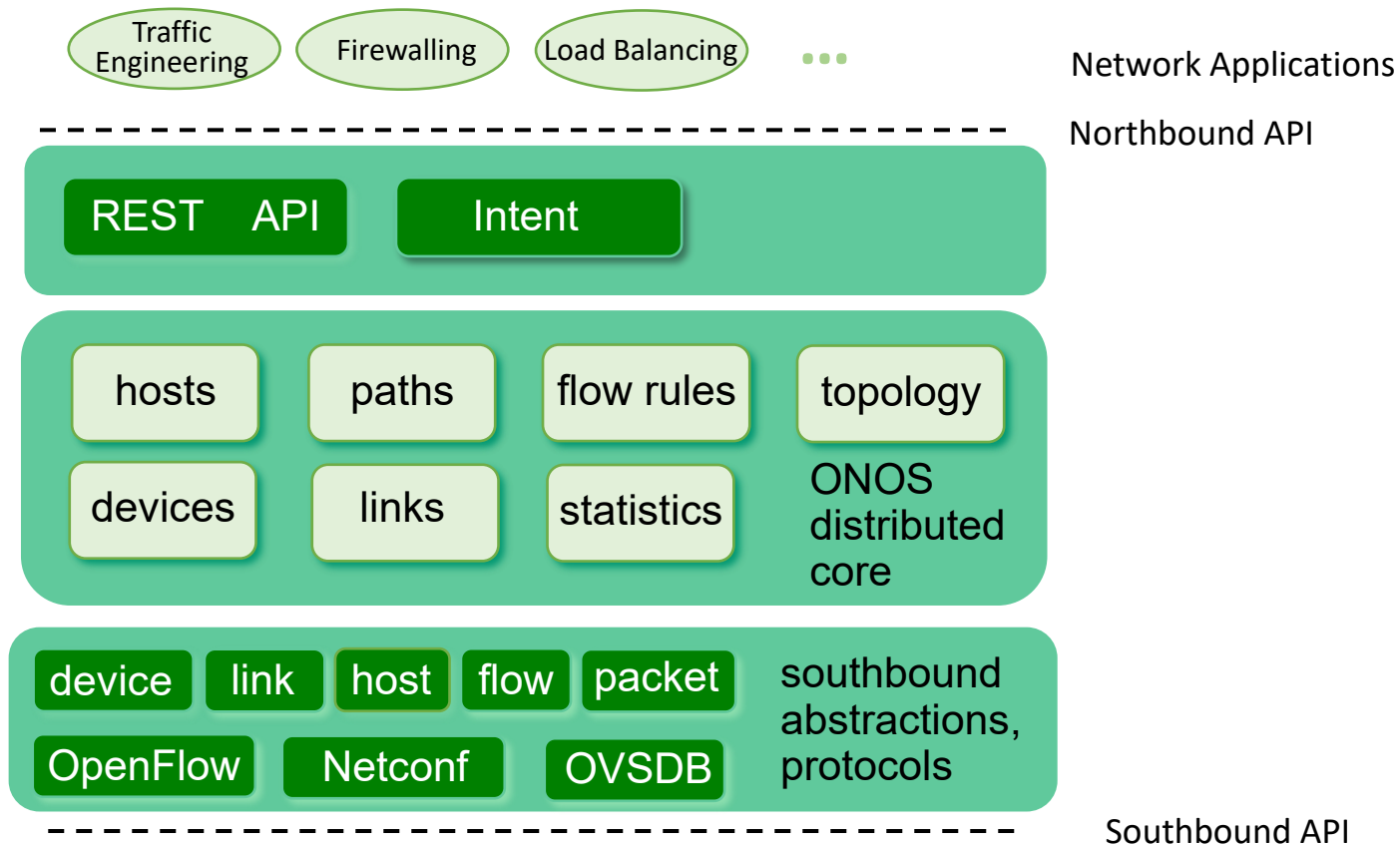
OpenDaylight (ODL) controller



Service Abstraction Layer:

- interconnects internal, external applications and services

ONOS controller



- control apps separate from controller
- intent framework: high-level specification of service: what rather than how
- considerable emphasis on distributed core: service reliability, replication performance scaling

SDN: selected challenges

- hardening the control plane: dependable, reliable, performance-scalable, secure distributed system
 - robustness to failures: leverage strong theory of reliable distributed system for control plane
 - dependability, security: “baked in” from day one?
- networks, protocols meeting mission-specific requirements
 - e.g., real-time, ultra-reliable, ultra-secure
- Internet-scaling: beyond a single AS
- SDN critical in 5G cellular networks

SDN and the future of traditional network protocols

- SDN-computed versus router-computer forwarding tables:
 - just one example of logically-centralized-computed versus protocol computed
- one could imagine SDN-computed congestion control:
 - controller sets sender rates based on router-reported (to controller) congestion levels



How will implementation of network functionality (SDN versus protocols) evolve?



Network layer: “control plane” roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- **Internet Control Message Protocol**



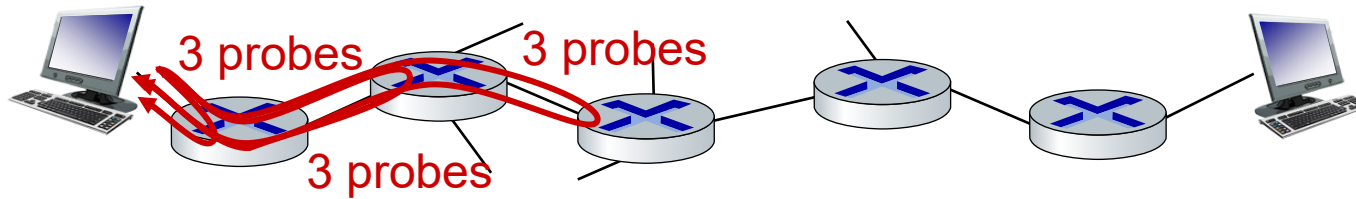
- network management, configuration
 - SNMP
 - NETCONF/YANG

ICMP: internet control message protocol

- used by hosts and routers to communicate network-level information
 - error reporting: unreachable host, network, port, protocol
 - echo request/reply (used by ping)
- network-layer “above” IP:
 - ICMP messages carried in IP datagrams
- *ICMP message*: type, code plus first 8 bytes of IP datagram causing error

<u>Type</u>	<u>Code</u>	<u>description</u>
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Traceroute and ICMP



- source sends sets of UDP segments to destination
 - 1st set has TTL =1, 2nd set has TTL=2, etc.
- datagram in n th set arrives to n th router:
 - router discards datagram and sends source ICMP message (type 11, code 0)
 - ICMP message possibly includes name of router & IP address
- when ICMP message arrives at source: record RTTs

stopping criteria:

- UDP segment eventually arrives at destination host
- destination returns ICMP “port unreachable” message (type 3, code 3)
- source stops

Network layer: “control plane” roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
 - SNMP
 - NETCONF/YANG

What is network management?

- autonomous systems (aka “network”): 1000s of interacting hardware/software components
- other complex systems requiring monitoring, configuration, control:
 - jet airplane, nuclear power plant, others?



"**Network management** includes the deployment, integration and coordination of the hardware, software, and human elements to monitor, test, poll, configure, analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost."

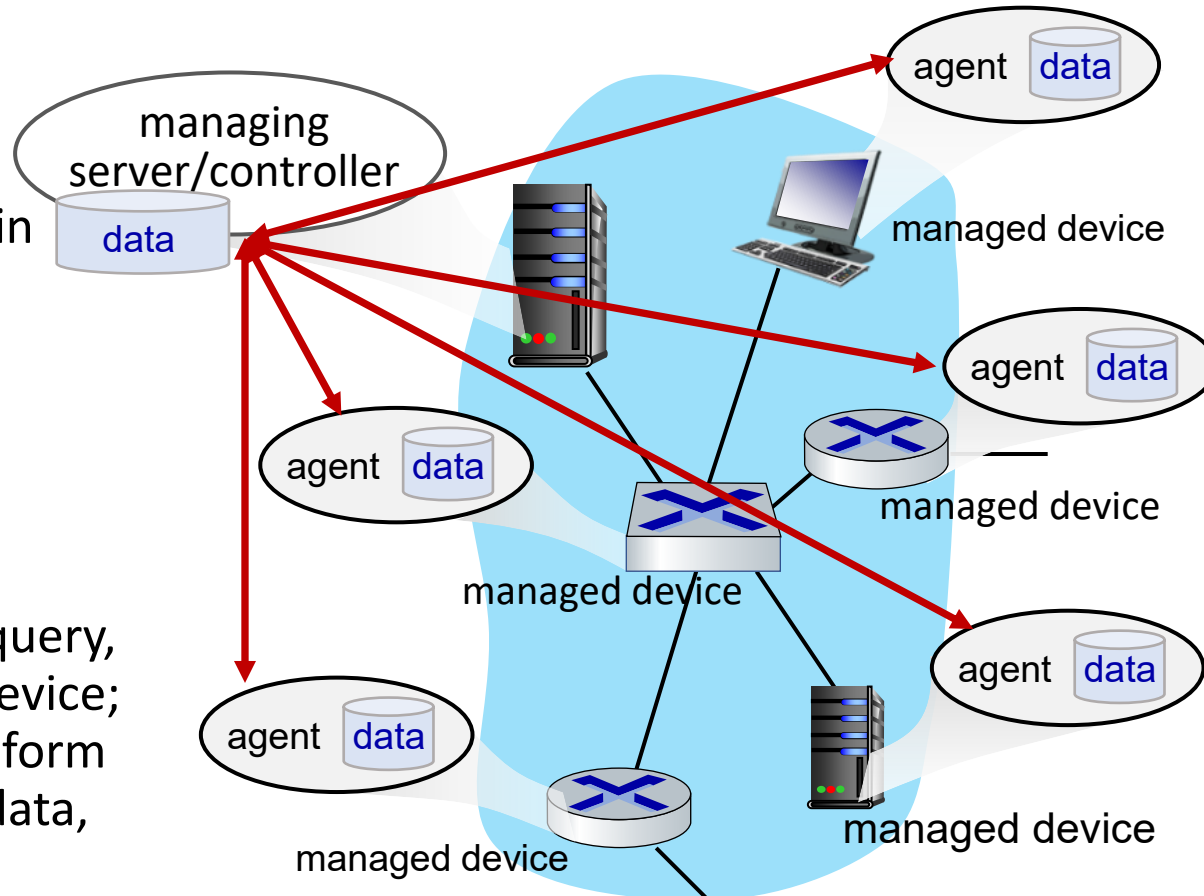
Components of network management

Managing server:

application, typically with network managers (humans) in the loop

Network management protocol:

used by managing server to query, configure, manage device; used by devices to inform managing server of data, events.



Managed device:

equipment with manageable, configurable hardware, software components

Data:

device “state”
configuration data,
operational data,
device statistics

Network operator approaches to management

CLI (Command Line Interface)

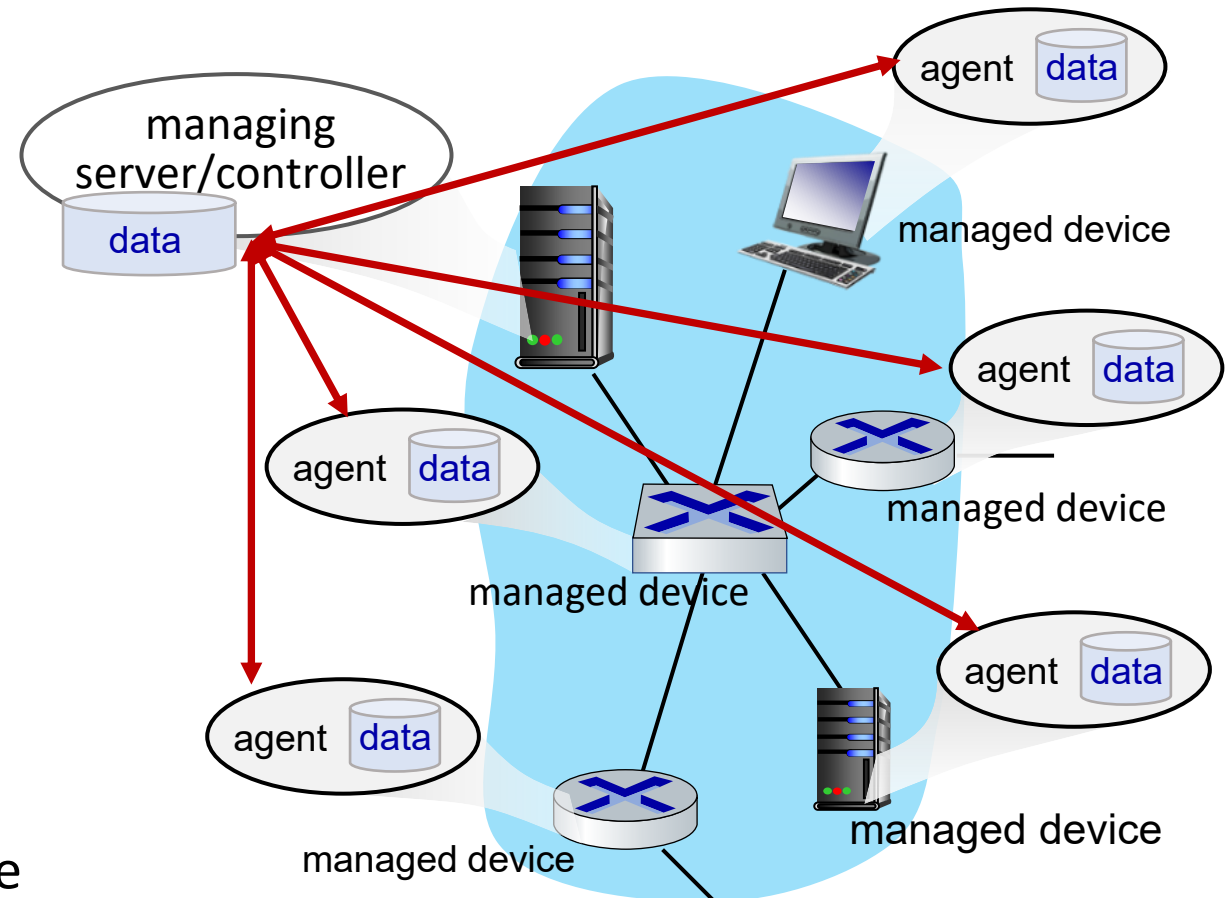
- operator issues (types, scripts) direct to individual devices (e.g., vis ssh)

SNMP/MIB

- operator queries/sets devices data (MIB) using Simple Network Management Protocol (SNMP)

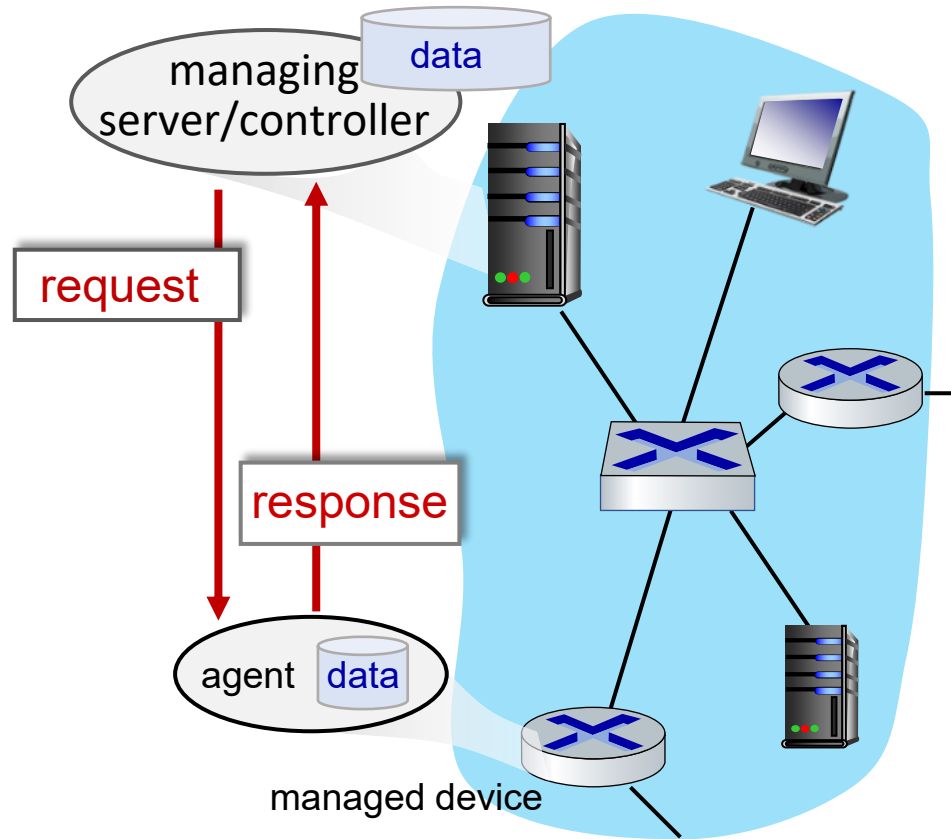
NETCONF/YANG

- more abstract, network-wide, holistic
- emphasis on multi-device configuration management.
- YANG: data modeling language
- NETCONF: communicate YANG-compatible actions/data to/from/among remote devices

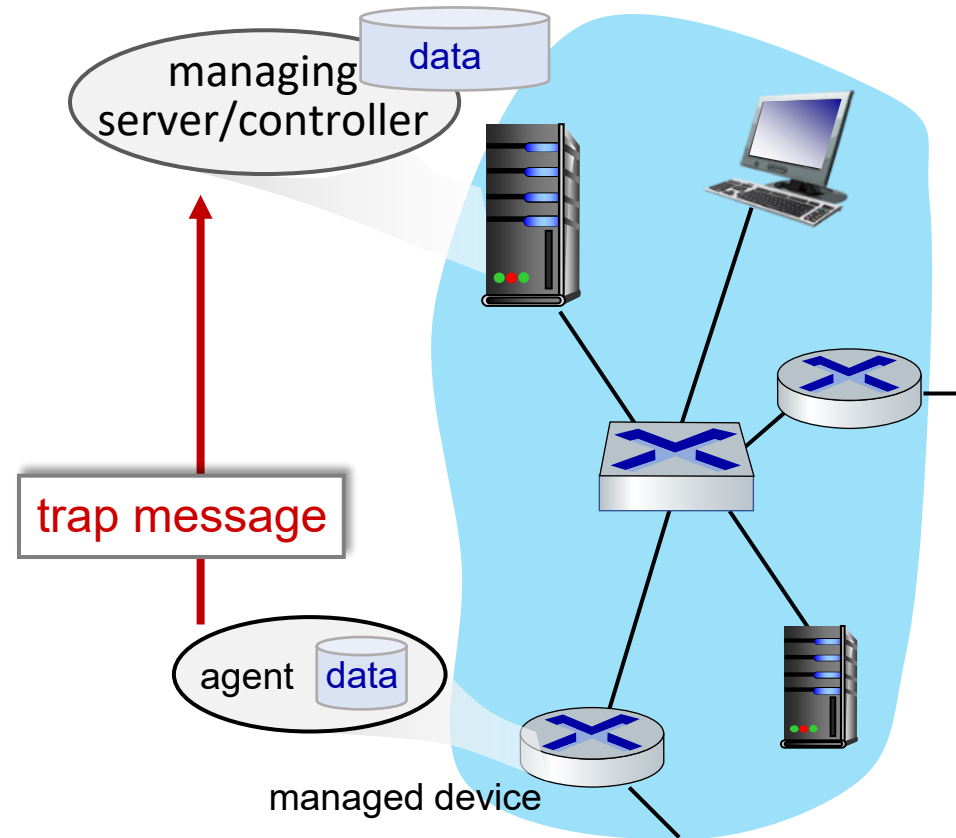


SNMP protocol

Two ways to convey MIB info, commands:



request/response mode



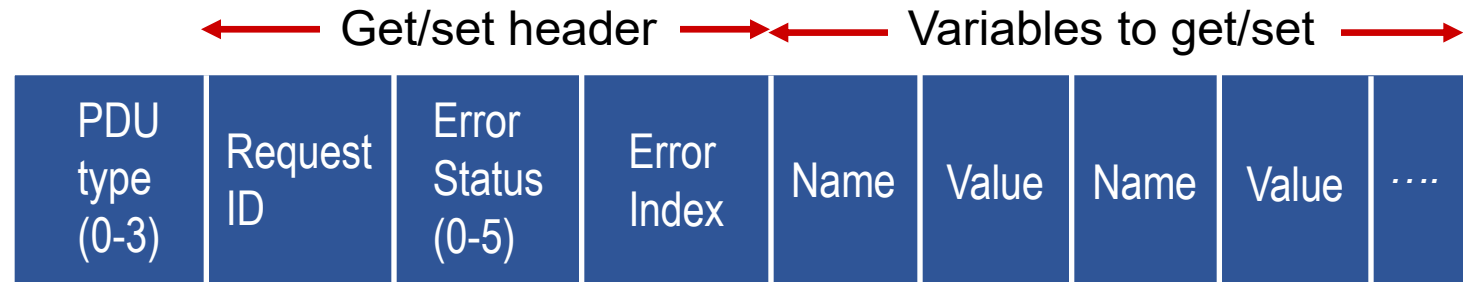
trap mode

SNMP protocol: message types

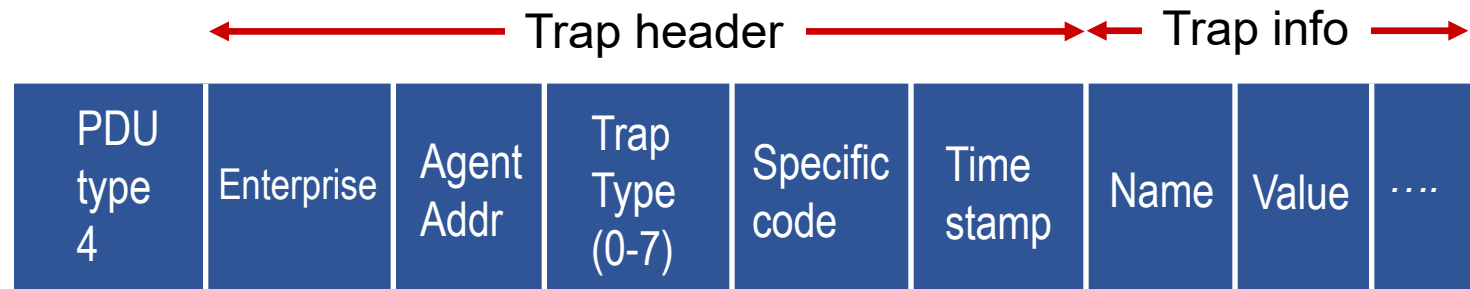
Message type	Function
GetRequest GetNextRequest GetBulkRequest	manager-to-agent: “get me data” (data instance, next data in list, block of data).
SetRequest	manager-to-agent: set MIB value
Response	Agent-to-manager: value, response to Request
Trap	Agent-to-manager: inform manager of exceptional event

SNMP protocol: message formats

message types 0-3

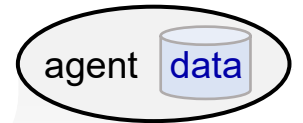


message type 4



SNMP: Management Information Base (MIB)

- managed device's operational (and some configuration) data
- gathered into device **MIB module**
 - 400 MIB modules defined in RFC's; many more vendor-specific MIBs
- **Structure of Management Information (SMI):** data definition language
- example MIB variables for UDP protocol:



Object ID	Name	Type	Comments
1.3.6.1.2.1.7.1	UDPInDatagrams	32-bit counter	total # datagrams delivered
1.3.6.1.2.1.7.2	UDPNoPorts	32-bit counter	# undeliverable datagrams (no application at port)
1.3.6.1.2.1.7.3	UDInErrors	32-bit counter	# undeliverable datagrams (all other reasons)
1.3.6.1.2.1.7.4	UDPOutDatagrams	32-bit counter	total # datagrams sent
1.3.6.1.2.1.7.5	udpTable	SEQUENCE	one entry for each port currently in use

NETCONF

- Network Configuration Protocol (NETCONF)
 - NETCONF is an application-layer protocol to provide network administrators with mechanisms to install, manipulate, and delete the configuration of network devices.
 - It uses an Extensible Markup Language (XML)-based data encoding for configuration data as well as protocol messages.
 - RFC 6241: <https://tools.ietf.org/html/rfc6241>
- **goal:** actively manage/**configure** devices network-wide
- operates between managing server and managed network devices
 - actions: retrieve, set, modify, activate configurations
 - **atomic-commit** actions over multiple devices
 - query operational data and statistics
 - subscribe to notifications from devices

NETCONF

- NETCONF's remote procedure call (RPC) paradigm
 - NETCONF protocol messages encoded in XML
 - exchanged over secure, reliable transport (e.g., TLS) protocol

- NETCONF RPC
 - A client encodes an RPC in XML and sends it to a server using a secure, connection-oriented session.
 - The server responds with a reply encoded in XML.
 - The contents of both the request and the response are fully described in XML DTDs (Document Type Definitions) or XML schemas, or both. The purpose of a DTD is to define the structure of an XML document.
 - The XML DTDs and XML schemas allow both parties to recognize the syntax constraints imposed on the exchange.

NETCONF

- http://www.e-cartouche.ch/content_reg/cartouche/datastruc/en/html/dtd_schema_learningObject1.html

```
<?xml version="1.0"?>
<!DOCTYPE person [
    <!ELEMENT person (age,gender,name)>
    <!ELEMENT age (#CDATA)>
    <!ELEMENT gender (#PCDATA)>
    <!ELEMENT name (#PCDATA)>
]>
```

← Document Type Definitions

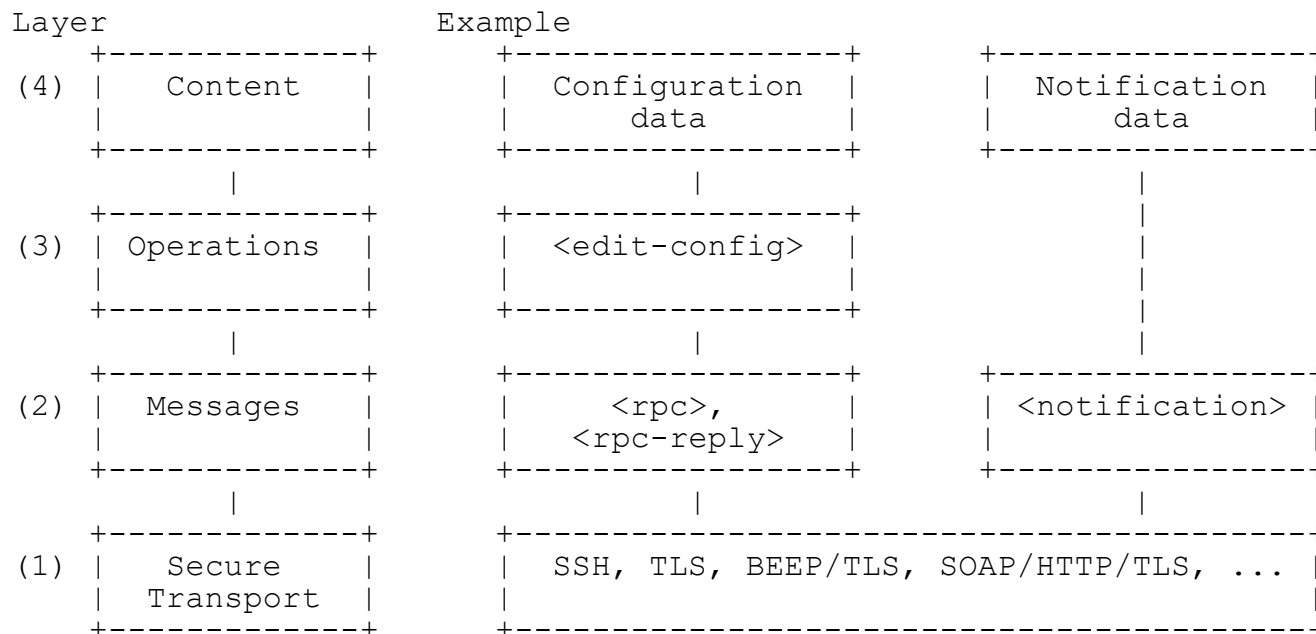
```
<person>
    <age>40</age>
    <gender>male</gender>
    <name>Peter Miller</name>
</person>
```

← XML Schema

NETCONF protocol (1/2)

Protocol Overview

- NETCONF uses a simple RPC-based mechanism to facilitate communication between a client and a server.
- The **client** can be a script or application typically running as part of a network manager.
- The **server** is typically a network device.



- 1) The **Secure Transport layer** provides a secure communication path between the client and server. NETCONF can be layered over any transport protocol that provides a set of basic requirements.
- 2) The **Messages layer** provides a simple, transport-independent framing mechanism for encoding RPCs and notifications.
- 3) The **Operations layer** defines a set of base protocol operations invoked as RPC methods with XML-encoded parameters.
- 4) The **Content layer** is the NETCONF data which constructed by a YANG data modeling language.

NETCONF protocol (2/2)

NETCONF RPC

- The `<rpc>` element invokes the NETCONF `<get>` method and includes an additional attribute called “user-id”.
- Note that the “user-id” attribute is not in the NETCONF’s namespace, but in the data model’s namespace.
- The returned `<rpc-reply>` element returns the “user-id” attribute, as well as the requested content.

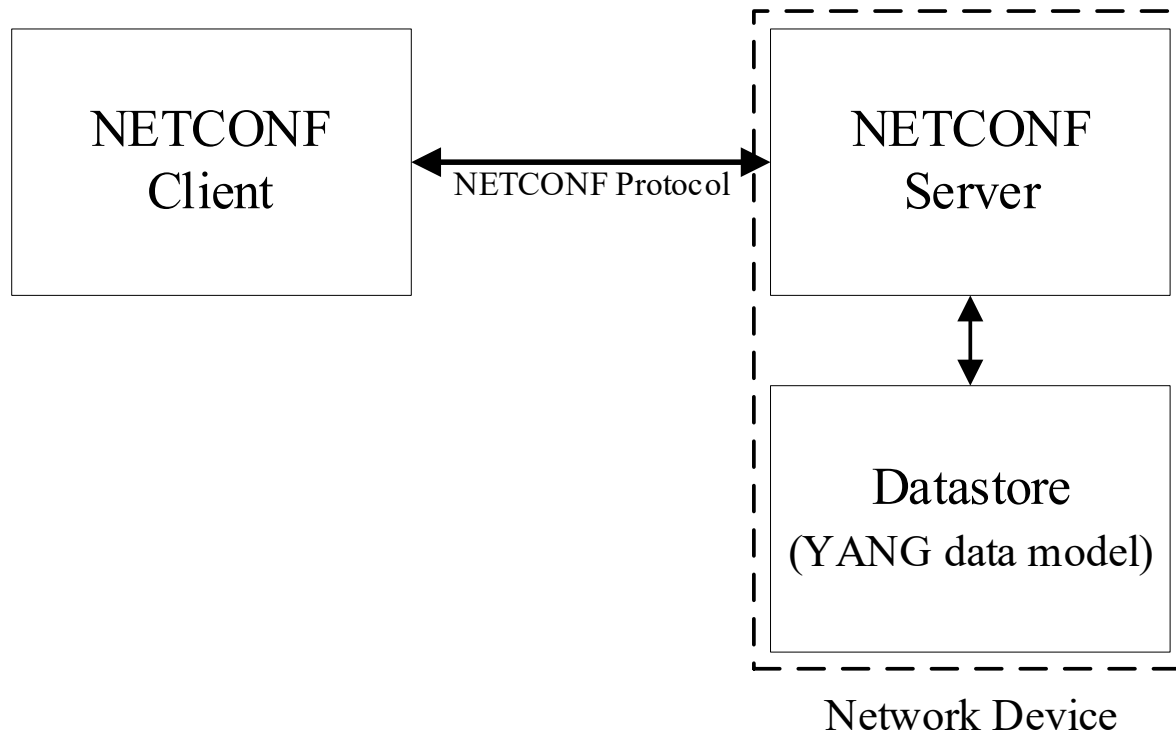
• RPC Request:

```
<rpc message-id="101"
  xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
  xmlns:ex="http://example.net/content/1.0"
  ex:user-id="fred">
  <get/>
</rpc>
```

• RPC Reply:

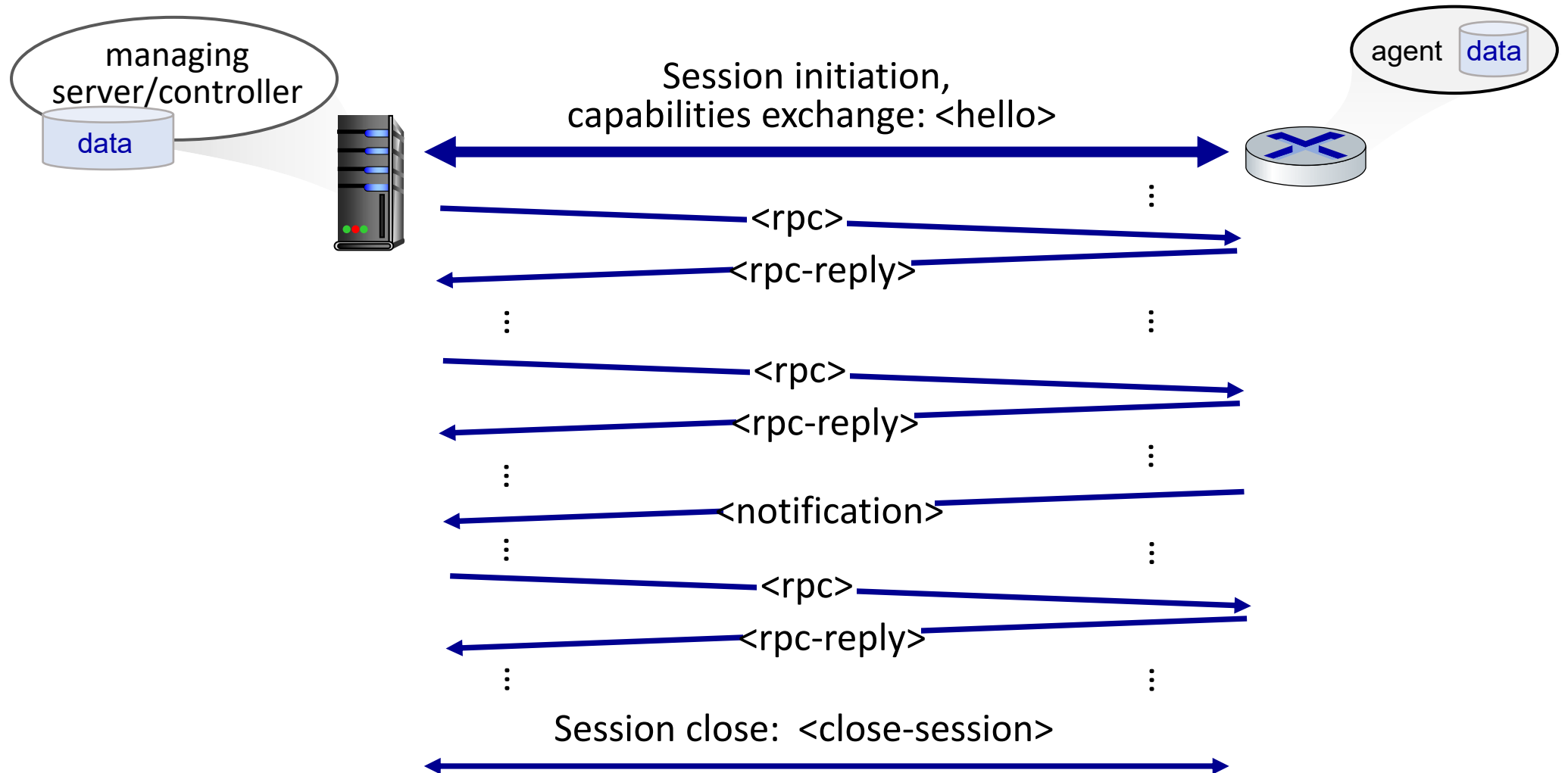
```
<rpc-reply message-id="101"
  xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
  xmlns:ex="http://example.net/content/1.0"
  ex:user-id="fred">
  <data>
    <!-- contents here... -->
  </data>
</rpc-reply>
```


NETCONF Components



- The NETCONF client sends RPC messages to the network device.
- The NETCONF server is running on the network device will interact with the datastore.
- The datastore contains configuration data and state data.
- The datastore is defined by YANG data models.

NETCONF initialization, exchange, close



Selected NETCONF Operations

NETCONF	Operation Description
<get-config>	Retrieve all or part of a given configuration. A device may have multiple configurations.
<get>	Retrieve all or part of both configuration state and operational state data.
<edit-config>	Change specified (possibly running) configuration at managed device. Managed device <rpc-reply> contains <ok> or <rpcerror> with rollback.
<lock>, <unlock>	Lock (unlock) configuration datastore at managed device (to lock out NETCONF, SNMP, or CLIs commands from other sources).
<create-subscription>, <notification>	Enable event notification subscription from managed device

Sample NETCONF RPC message

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <rpc message-id="101"  note message id
03   xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
04   <edit-config>      change a configuration
05     <target>
06       <running/>  change the running configuration
07     </target>
08     <config>
09       <top xmlns="http://example.com/schema/
10         1.2/config">
11         <interface>
12           <name>Ethernet0/0</name>  change MTU of Ethernet 0/0 interface to 1500
13           <mtu>1500</mtu>
14         </interface>
15       </top>
16     </config>
17   </edit-config>
18 </rpc>
```

RESTCONF

- RESTCONF is an **HTTP-based protocol** that provides a programmatic interface for accessing data defined in YANG, using the datastore concepts defined in NETCONF in RFC 8040.
 - RFC 8040: <https://tools.ietf.org/html/rfc8040>
- NETCONF defines configuration datastores and a set of **Create, Read, Update, Delete (CRUD) operations** that can be used to access these datastores.
 - RESTCONF uses HTTP methods to provide CRUD operations on a datastore containing YANG-defined data.

Why RESTCONF?

If RESTCONF and NETCONF are similar, why do we need RESTCONF?

- Because of the trend of the IT industry. The most popular protocol used in the modern era for automation is HTTP-based REST APIs.
- Most software developers know REST APIs, but do not know NETCONF.
 - If the developers need to create an automation system for their networking devices, they can use RESTCONF instead of learning a new protocol, i.e., NETCONF.
- These demands push the creation of RESTCONF.

RESTCONF Methods

- The RESTCONF protocol uses HTTP methods to identify the **CRUD operations** requested for a particular resource.
- The following table shows how the RESTCONF operations relate to NETCONF protocol operations.

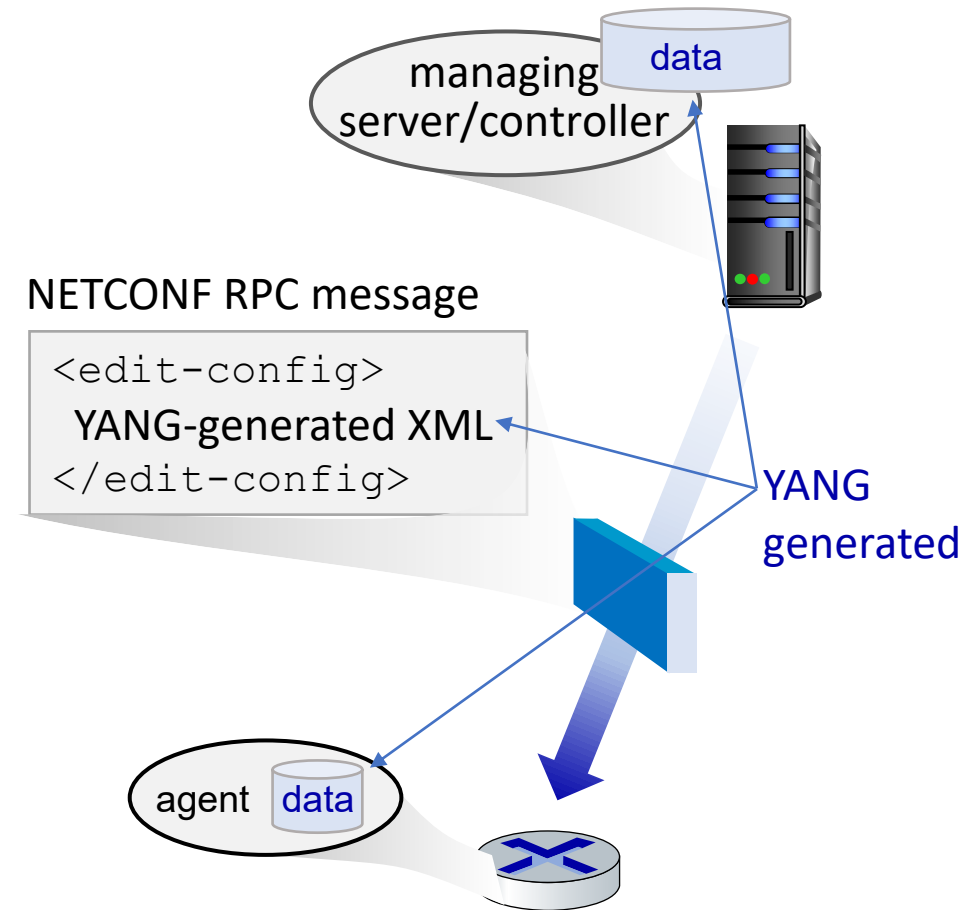
RESTCONF	NETCONF
OPTIONS	none
HEAD	<get-config>, <get>
GET	<get-config>, <get>
POST (Create Resource Mode)	<edit-config>
POST (Invoke Operation Mode)	Invoke an RPC operation
PUT	<copy-config> (PUT on datastore)
PUT	<edit-config> (nc:operation="create/replace")
PATCH	<edit-config> (nc:operation depends on PATCH content)
DELETE	<edit-config> (nc:operation="delete")

NETCONF vs. RESTCONF

Category	NETCONF	RESTCONF
CRUD Operations	Wrapping the message content in an XML tag, e.g., <code><get-config></code> , <code><edit-config></code>	HTTP method, e.g., GET, POST, PUT, PATCH, DELETE
Data Encoding Format	RPC messages encoded with XML	Data can be encoded with either XML or JSON
Transmission Channel	Prefer SSH at the transport layer and use SSH to transmit XML information	Access a device based on HTTP or HTTPS. Comply with RESTful style.
Configuration phase	Two phase configuration validation. Parameters are configured and committed before taking effect.	Operations take effect immediately, without the need to be submitted in two phases.
Tools	Python ncclient and SSH with CLI	cURL, Requests python, and Postman

YANG

- Yet Another Next Generation (YANG) data modeling language
 - RFC 7950: <https://www.rfc-editor.org/rfc/rfc7950>
- YANG is used to specify structure, syntax, semantics of NETCONF network management data
 - built-in data types, like SMI
- XML document describing device capabilities can be generated from YANG description
- can express constraints among data that must be satisfied by a valid NETCONF configuration
 - ensure NETCONF configurations satisfy correctness, consistency constraints



YANG Functional Overview (1/5)

- YANG is a language used to model data for the NETCONF protocol.
- A YANG module defines a hierarchy of data that can be used for NETCONF-based operations such as
 - configuration, state data, Remote Procedure Calls (RPCs) and notifications.
- YANG models the hierarchical organization of data as a tree:
 - Each node has a name, and either a value or a set of child nodes.
- Augmentable hierarchy
 - One module is allowed to add data nodes to the hierarchy defined in another module.
 - This augmentation can be conditional with new nodes appearing only if certain conditions are met.

YANG Functional Overview (2/5)

■ Constraint enforcement

- YANG models can describe constraints to be enforced on the data.
- They restrict the appearance or value of nodes based on the presence or value of other nodes in the hierarchy.
- These constraints are enforceable by either the NETCONF client or the NETCONF server.

■ Data types

- YANG defines a set of built-in types.
- It has type mechanism through which additional types may be defined.
- Derived types can restrict their base type's set of valid values using mechanisms like range or pattern restrictions
- It can define usage conventions for use of the derived type, such as a string-based type that contains a host name.

YANG Functional Overview (3/5)

- Grouping
 - YANG grouping allows for reusable groupings of nodes.
 - The instantiation of the groupings can refine or augment the nodes, allowing it to tailor the nodes to its particular needs.
- List
 - List entries are identified by keys.
 - Lists may be defined as either sorted by user or automatically sorted by the system.
- YANG Independent Notation (YIN)
 - YANG modules can be translated into an equivalent XML syntax called YANG Independent Notation (YIN).
 - This translation allows applications using XML parsers and Extensible Stylesheet Language Transformations (XSLT) scripts to operate on the models.
 - YANG (陽) ⇔ YIN (陰): Lossless conversion between YANG and YIN.

YANG Functional Overview (4/5)

■ The Goals of YANG

- YANG supports the translation from high-level data modeling to low-level bits-on-the-wire encoding.
 - A reader of a YANG module can see the high-level view of data model while understanding how the data will be encoded in NETCONF operations.
- YANG is an extensible language, allowing extension statements to be defined by standard bodies, vendors, and individuals.
- YANG aims at the problem space to allow expression of NETCONF data models rather than arbitrary XML documents or arbitrary data models.
 - The data models described by YANG are designed to be easily operated upon by NETCONF operations.

YANG Functional Overview (5/5)

- Compatibility with Simple Network Management Protocol (SNMP)
 - YANG maintains compatibility with SNMP's SMIv2 (Structure of Management Information version 2).
 - SMIv2-based MIB (Management Information Base) modules can be translated into YANG modules for read-only access.

- Device Management
 - Like NETCONF, YANG targets smooth integration with a device's native management infrastructure.
 - This allows implementations to leverage their existing access control mechanism to protect or expose elements of the data model.

Network layer: Summary

we've learned a lot!

- approaches to network control plane
 - per-router control (traditional)
 - logically centralized control (software defined networking)
- traditional routing algorithms
 - implementation in Internet: OSPF, BGP
- SDN controllers
 - implementation in practice: ODL, ONOS
- Internet Control Message Protocol
- network management

next stop: link layer!

Network layer, control plane: Done!

- introduction
- routing protocols
 - link state
 - distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

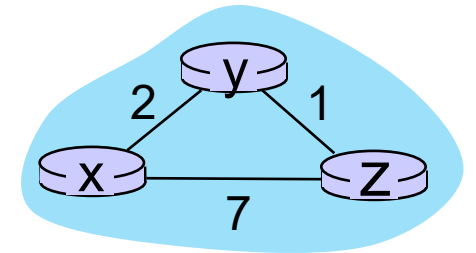
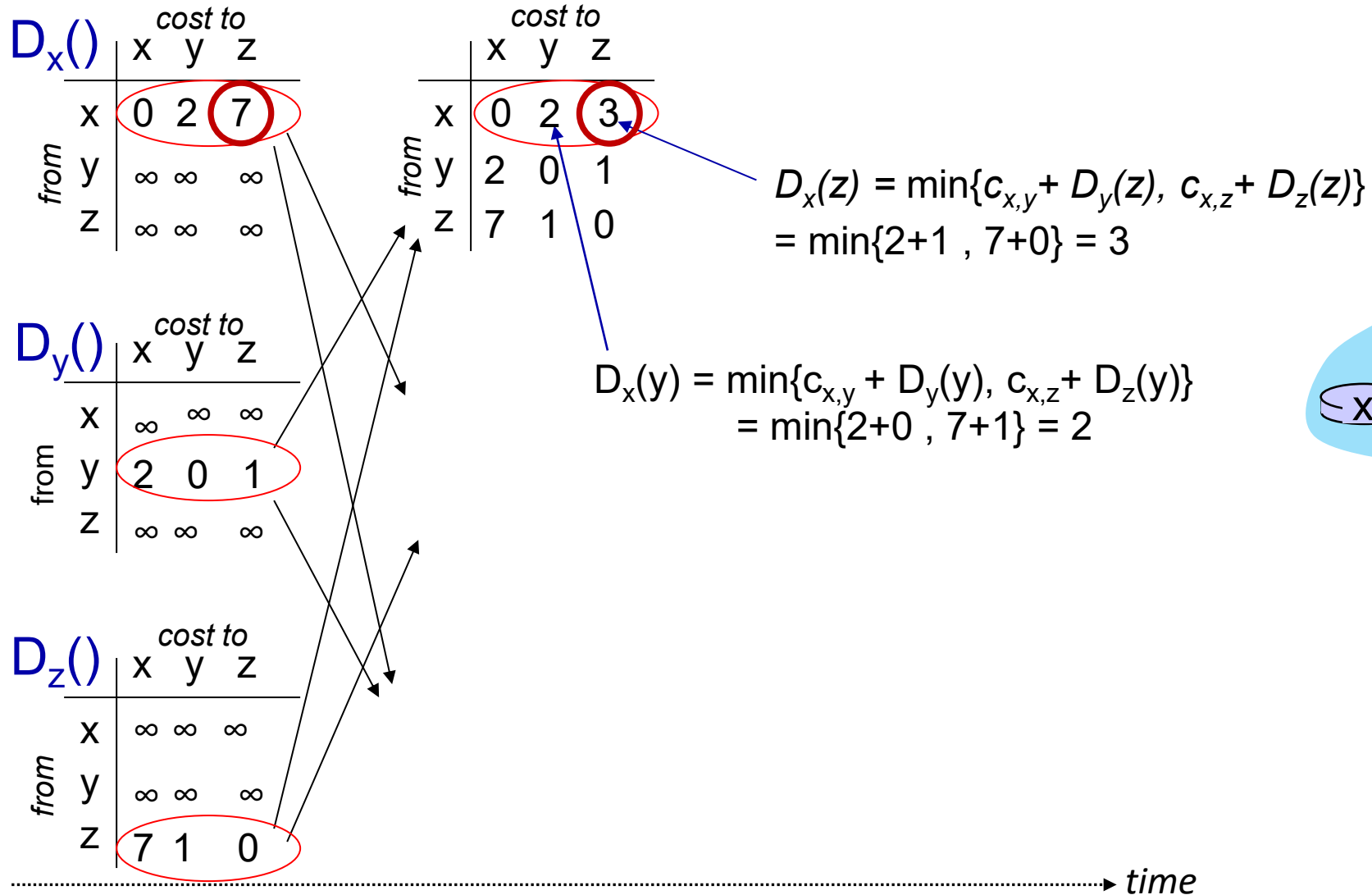


- network management, configuration
 - SNMP
 - NETCONF/YANG

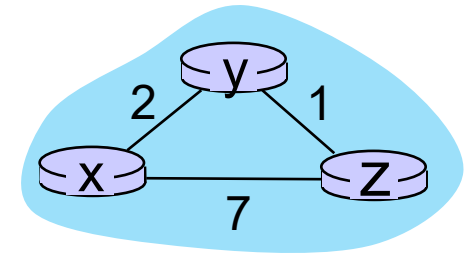
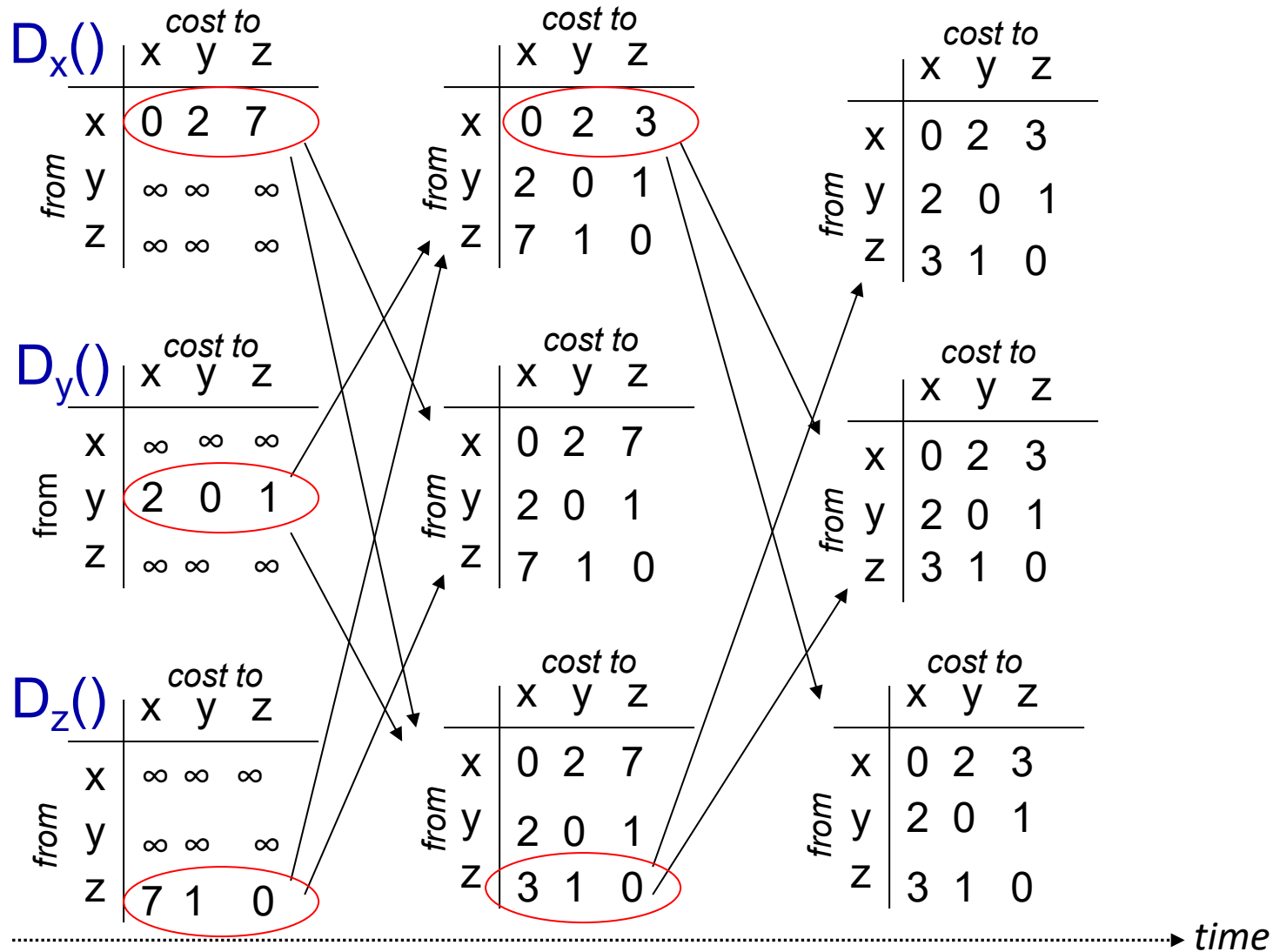
Appendices

Additional Chapter 5 slides

Distance vector: another example



Distance vector: another example



YANG Tutorial

M. Bjorklund, "The YANG 1.1 Data Modeling Language", RFC 7950.
<https://www.rfc-editor.org/rfc/rfc7950>

Language Overview (1/21)

■ Modules and Submodules

- A module contains three types of statements such as module-header statements, revision-statements, and definition statements.
- The module header statements describe the module and give information about the module itself.
- The revision statements give information about the history of the module.
- The definition statements are the body of the module where the data model is defined.
- A module may be divided into submodules, based on the needs of the module owner.
 - The external view remains that of a single module, regardless of the presence or size of its submodules.
- The “include” statement allows a module or submodule to reference material in submodules.
- The “import” statement allows references to material defined in other modules.

Language Overview (2/21)

- Data Modeling Basics

- YANG defines four main types of data nodes for data modeling:
 1. Leaf Nodes
 2. Leaf-List Nodes
 3. Container Nodes
 4. List Nodes

Language Overview (3/21)

■ Leaf Nodes

- A leaf node contains simple data like an integer or a string.
- It has exactly one value of a particular type and no child nodes.

- YANG Example:

```
leaf host-name {  
    type string;  
    description "Hostname for this system";  
}
```

- NETCONF XML Example:

```
<host-name>my.example.com</host-name>
```


Language Overview (4/21)

■ Leaf-List Nodes

- A leaf-list is a sequence of leaf nodes with exactly one value of a particular type per leaf.

- YANG Example:

```
leaf-list domain-search {  
    type string;  
    description "List of domain names to search";  
}
```

- NETCONF XML Example:

```
<domain-search>high.example.com</domain-search>  
<domain-search>low.example.com</domain-search>  
<domain-search>everywhere.example.com</domain-search>
```

Language Overview (5/21)

■ Container Nodes

- A container node is used to group related nodes in a subtree.
- A container has only child nodes and no value.
- A container may contain any number of child nodes of any type (e.g., leafs, lists, containers, and leaf-lists).
- A container takes one argument, which is an identifier, followed by a block of substatements that holds detailed container information.

• YANG Example:

```
container system {  
    container login {  
        leaf message {  
            type string;  
            description "Message given at start of login session";  
        }  
    }  
}
```

• NETCONF XML Example:

```
<system>  
    <login>  
        <message>Good morning</message>  
    </login>  
</system>
```

Language Overview (6/21)

■ List Nodes

- A list defines a sequence of list entries.
- Each entry is like a structure or a record instance.
- Each entry is uniquely identified by the values of its key leafs.
- A list can define multiple key leafs and may contain any number of child nodes of any type (e.g., leafs, lists, containers, etc.).

• YANG Example:

```
list user {  
    key "name";  
    leaf name {  
        type string;  
    }  
    leaf full-name {  
        type string;  
    }  
    leaf class {  
        type string;  
    }  
}
```

• NETCONF XML Example:

```
<user>  
    <name>glocks</name>  
    <full-name>Goldie Locks</full-name>  
    <class>intruder</class>  
</user>  
<user>  
    <name>snowy</name>  
    <full-name>Snow White</full-name>  
    <class>free-loader</class>  
</user>
```

Language Overview (7/21)

■ Example of Full YANG Module

- These statements (i.e., Leaf, Leaf-List, Container, List) are combined to define a YANG module.
- Example YANG Module for ACME (Automated Certificate Management Environment).

```
// Contents of "example-system.yang"
module example-system {
  yang-version 1.1;
  namespace "urn:example:system";
  prefix "sys";

  organization "Example Inc.";
  contact "joe@example.com";
  description
    "The module for entities implementing the Example
    system.";

  revision 2007-06-09 {
    description "Initial revision.";
  }

  container system {
    leaf host-name {
      type string;
      description
        "Hostname for this system.";
    }

    leaf-list domain-search {
      type string;
      description
        "List of domain names to search.";
    }

    container login {
      leaf message {
        type string;
        description
          "Message given at start of login session.";
      }

      list user {
        key "name";
        leaf name {
          type string;
        }
        leaf full-name {
          type string;
        }
        leaf class {
          type string;
        }
      }
    }
  }
}
```

Language Overview (8/21)

■ State Data

- YANG can model state data as well as configuration data, based on the “config” statement.
- When a node is tagged with "`config false`", its subhierarchy is flagged as state data.
- The state data can be reported using NETCONF's `<get>` operation, not the `<get-config>` operation.
- Parent containers, lists, and key leafs are also reported, giving the context for the state data.

Language Overview (9/21)

- The following example defines an interface with three leafs such as **name**, **speed**, and **observed-speed**.
 - **speed** is configuration, but **observed-speed** is not configuration.
 - **observed-speed** can be returned with NETCONF <get> operations, but not with <get-config> operations.
 - **observed-speed** cannot be manipulated using <edit-config>, either.

■ State Data (Con't)

- YANG Example:

```
list interface {  
    key "name";  
  
    leaf name {  
        type string;  
    }  
    leaf speed {  
        type enumeration {  
            enum 10m;  
            enum 100m;  
            enum auto;  
        }  
    }  
    leaf observed-speed {  
        type uint32;  
        config false;  
    }  
}
```

Language Overview (10/21)

■ Built-In Types

- YANG has a set of built-in types like programming languages.
- It has some differences due to special requirements from the management domain.
- The following table summarizes the built-in types:

Name	Description
binary	Any binary data
bits	A set of bits or flags
boolean	"true" or "false"
decimal64	64-bit signed decimal number
empty	A leaf that does not have any value
enumeration	One of an enumerated set of strings
identityref	A reference to an abstract identity
instance-identifier	A reference to a data tree node
int8	8-bit signed integer
int16	16-bit signed integer
int32	32-bit signed integer
int64	64-bit signed integer
leafref	A reference to a leaf instance
string	A character string
uint8	8-bit unsigned integer
uint16	16-bit unsigned integer
uint32	32-bit unsigned integer
uint64	64-bit unsigned integer
union	Choice of member types

Language Overview (11/21)

■ Derived Types (typedef)

- YANG can define derived types from base types using the “typedef” statement.
- A base type can be either a built-in type or a derived type, allowing a hierarchy of derived types.

YANG Example:

```
typedef percent {  
    type uint8 {  
        range "0 .. 100";  
    }  
    description "Percentage";  
}  
  
leaf completed {  
    type percent;  
}
```

NETCONF XML Example:

Valid value:

✓ **<completed>20</completed>**

Invalid value:

✗ **<completed>101</completed>**

Language Overview (12/21)

■ Derived Types (typedef) (Con't)

- RFC 6991 introduces a collection of common data types to be used with the YANG data modeling language.

YANG type	Equivalent SMIV2 type (module)
counter32	Counter32 (SNMPv2-SMI)
zero-based-counter32	ZeroBasedCounter32 (RMON2-MIB)
counter64	Counter64 (SNMPv2-SMI)
zero-based-counter64	ZeroBasedCounter64 (HCNUM-TC)
gauge32	Gauge32 (SNMPv2-SMI)
gauge64	CounterBasedGauge64 (HCNUM-TC)
object-identifier	-
object-identifier-128	OBJECT IDENTIFIER
yang-identifier	-
date-and-time	-
timeticks	TimeTicks (SNMPv2-SMI)
timestamp	TimeStamp (SNMPv2-TC)
phys-address	PhysAddress (SNMPv2-TC)
mac-address	MacAddress (SNMPv2-TC)
xpath1.0	-
hex-string	-
uuid	-
dotted-quad	-

Table 1: ietf-yang-types

YANG type	Equivalent SMIV2 type (module)
ip-version	InetVersion (INET-ADDRESS-MIB)
dscp	Dscp (DIFFSERV-DSCP-TC)
ipv6-flow-label	IPv6FlowLabel (IPV6-FLOW-LABEL-MIB)
port-number	InetPortNumber (INET-ADDRESS-MIB)
as-number	InetAutonomousSystemNumber (INET-ADDRESS-MIB)
ip-address	-
ipv4-address	-
ipv6-address	-
ip-address-no-zone	-
ipv4-address-no-zone	-
ipv6-address-no-zone	-
ip-prefix	-
ipv4-prefix	-
ipv6-prefix	-
domain-name	-
host	-
uri	Uri (URI-TC-MIB)

Table 2: ietf-inet-types

Language Overview (13/21)

■ Derived Types (typedef) (Con't)

- Examples of the derived types in RFC 6991:

```
typedef ipv4-prefix {  
    type string {  
        pattern  
            '(([0-9]|[1-9][0-9]|1[0-9][0-9]|2[0-4][0-9]|25[0-5])\.){3}'  
            + '([0-9]|[1-9][0-9]|1[0-9][0-9]|2[0-4][0-9]|25[0-5])'  
            + '/((([0-9])|([1-2][0-9])|([3][0-2])))';  
    }  
    description  
        "The ipv4-prefix type represents an IPv4 address prefix.  
        The prefix length is given by the number following the  
        slash character and must be less than or equal to 32.  
  
        A prefix length value of n corresponds to an IP address  
        mask that has n contiguous 1-bits from the most  
        significant bit (MSB) and all other bits set to 0.  
  
        The canonical format of an IPv4 prefix has all bits of  
        the IPv4 address set to zero that are not part of the  
        IPv4 prefix.";  
}
```

```
typedef ipv6-prefix {  
    type string {  
        pattern '(:|([0-9a-fA-F]{0,4}):)([0-9a-fA-F]{0,4}:){0,5}'  
            + '(((([0-9a-fA-F]{0,4}):)?(:|([0-9a-fA-F]{0,4})))|'  
            + '((25[0-5]|2[0-4][0-9]|01?[0-9]?[0-9])\.){3}'  
            + '(25[0-5]|2[0-4][0-9]|01?[0-9]?[0-9])))'  
            + '(/((([0-9])|([0-9]{2})|(1[0-1][0-9])|(12[0-8]))))';  
        pattern '(([[:]]+){6}([[:]]+:[[:]]+)|(.*\..*))|'  
            + '((([:]]+)*[[:]]+)?::([[:]]+)*[[:]]+)?|'  
            + '(/.+)';  
    }  
    description  
        "The ipv6-prefix type represents an IPv6 address prefix.  
        The prefix length is given by the number following the  
        slash character and must be less than or equal to 128.  
  
        A prefix length value of n corresponds to an IP address  
        mask that has n contiguous 1-bits from the most  
        significant bit (MSB) and all other bits set to 0.  
  
        The IPv6 address should have all bits that do not belong  
        to the prefix set to zero.  
  
        The canonical format of an IPv6 prefix has all bits of  
        the IPv6 address set to zero that are not part of the  
        IPv6 prefix. Furthermore, the IPv6 address is represented  
        as defined in Section 4 of RFC 5952.";  
    reference  
        "RFC 5952: A Recommendation for IPv6 Address Text  
        Representation";  
}
```

Language Overview (14/21)

- Reusable Node Groups (grouping)
 - Groups of nodes can be assembled into reusable collections using the “grouping” statement.
 - A grouping defines a set of nodes that are instantiated with the “uses” statement:

YANG Example:

```
grouping target {  
  leaf address {  
    type inet:ip-address;  
    description "Target IP address";  
  }  
  leaf port {  
    type inet:port-number;  
    description "Target port number";  
  }  
}  
  
container peer {  
  container destination {  
    uses target;  
  }  
}
```

NETCONF XML Example:

```
<peer>  
  <destination>  
    <address>192.0.2.1</address>  
    <port>830</port>  
  </destination>  
</peer>
```

Language Overview (15/21)

- Reusable Node Groups (grouping) (Con't)
 - The grouping can be refined as it is used, allowing certain statements to be overridden.

YANG Example using “refine” to refine description:

```
container connection {
  container source {
    uses target {
      refine "address" {
        description "Source IP address";
      }
      refine "port" {
        description "Source port number";
      }
    }
  }
}
container destination {
  uses target {
    refine "address" {
      description "Destination IP address";
    }
    refine "port" {
      description "Destination port number";
    }
  }
}
```

Language Overview (16/21)

■ Choices

- YANG allows the data model to segregate incompatible nodes into distinct choices using the “choice” and “case” statements.
- The “choice” statement contains a set of “case” statements that define sets of schema nodes that cannot appear together.
- Each “case” may contain multiple nodes, but each node may appear in only one “case” under a “choice”.
- The choice and case nodes appear only in the schema tree, not in the data tree or NETCONF messages.
- When an element from one case is created, all elements from all other cases are implicitly deleted.

Language Overview (17/21)

■ Choices (Con't)

Choices YANG Example:

```
container food {  
  choice snack {  
    case sports-arena {  
      leaf pretzel {  
        type empty;  
      }  
      leaf beer {  
        type empty;  
      }  
    }  
    case late-night {  
      leaf chocolate {  
        type enumeration {  
          enum dark;  
          enum milk;  
          enum first-available;  
        }  
      }  
    }  
  }  
}
```

NETCONF XML Example:

Valid example:

```
<food>  
  <pretzel />  
  <beer />  
</food>
```

OR

```
<food>  
  <chocolate>milk<chocolate/>  
</food>
```

Invalid example:

```
<food>  
  <pretzel />  
  <chocolate>milk<chocolate/>  
</food>
```

Language Overview (18/21)

- Extending Data Models (augment)
 - YANG allows a module to insert additional nodes into data models, including both the current module (and its submodules) or an external module.
 - For example, this is useful for vendors to add vendor-specific parameters to standard data models in an interoperable way.
 - The “augment” statement defines the location in the data model hierarchy where new nodes are inserted.
 - The “when” statement defines the conditions when the new nodes are valid.

Language Overview (19/21)

■ Extending Data Models (augment) (Con't)

- YANG Example, Extending from the **example-system** YANG module (Refer to slide [23](#)):

```
augment /system/login/user {  
  when "class != 'wheel'";  
  leaf uid {  
    type uint16 {  
      range "1000 .. 30000";  
    }  
  }  
}
```

■ Extending Data Models (augment) (Con't)

- NETCONF XML Example:

```
<user>  
  <name>alice</name>  
  <full-name>Alice N. Lee</full-name>  
  ✓ <class>drop-out</class>  
  <other:uid>1024</other:uid>  
</user>
```

```
<user>  
  <name>bobby</name>  
  <full-name>Bob M. Ree</full-name>  
  ✗ <class>wheel</class>  
  <other:uid>1025</other:uid>  
</user>
```


Language Overview (20/21)

■ RPC Definitions

- YANG allows the definition of NETCONF RPCs.
- The operations' names, input parameters, and output parameters are modeled using YANG data definition statements.

YANG Example:

```
rpc activate-software-image {  
  input {  
    leaf image-name {  
      type string;  
    }  
  }  
  output {  
    leaf status {  
      type string;  
    }  
  }  
}
```

NETCONF Example:

```
<rpc message-id="101"  
  xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">  
  <activate-software-image xmlns="http://acme.example.com/system">  
    <image-name>acmefw-2.3</image-name>  
  </activate-software-image>  
</rpc>  
  
<rpc-reply message-id="101"  
  xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">  
  <status xmlns="http://acme.example.com/system">  
    The image acmefw-2.3 is being installed.  
  </status>  
</rpc-reply>
```

Language Overview (21/21)

■ Notification Definitions

- YANG allows the definition of notifications suitable for NETCONF.
- YANG data definition statements are used to model the content of the notification.

YANG Example:

```
notification link-failure {  
  description "A link failure has been detected";  
  leaf if-name {  
    type leafref {  
      path "/interface/name";  
    }  
  }  
  leaf if-admin-status {  
    type admin-status;  
  }  
  leaf if-oper-status {  
    type oper-status;  
  }  
}
```

NETCONF XML Example:

```
<notification  
  xmlns="urn:ietf:params:netconf:capability:notification:1.0">  
  <eventTime>2007-09-01T10:00:00Z</eventTime>  
  <link-failure xmlns="http://acme.example.com/system">  
    <if-name>so-1/2/3.0</if-name>  
    <if-admin-status>up</if-admin-status>  
    <if-oper-status>down</if-oper-status>  
  </link-failure>  
</notification>
```

YANG Tree Diagram

- YANG Tree Diagram is a **simplified graphical representation** of a YANG data model and can be automatically generated via tools such as "pyang" [[PYANG](#)].
- [RFC 8340](#) (YANG Tree Diagrams) describes the current syntax used in YANG module tree diagrams.
- The "example-system" YANG module shown on the right can be simplified as the YANG tree diagram shown below:

```
module: example-system
  +--rw system
    +--rw host-name?      string
    +--rw domain-search*  string
    +--rw login
      +--rw message?      string
      +--rw user* [name]
        +--rw name          string
        +--rw full-name?    string
        +--rw class?        string
```

```
// Contents of "example-system.yang"
module example-system {
  yang-version 1.1;
  namespace "urn:example:system";
  prefix "sys";

  organization "Example Inc.";
  contact "joe@example.com";
  description
    "The module for entities implementing the Example
    system.";

  revision 2007-06-09 {
    description "Initial revision.";
  }

  container system {
    leaf host-name {
      type string;
      description
        "Hostname for this system.";
    }

    leaf-list domain-search {
      type string;
      description
        "List of domain names to search.";
    }

    container login {
      leaf message {
        type string;
        description
          "Message given at start of login session.";
      }
      list user {
        key "name";
        leaf name {
          type string;
        }
        leaf full-name {
          type string;
        }
        leaf class {
          type string;
        }
      }
    }
  }
}
```

YANG Tree Diagram Syntax (1/3)

- Each node in a YANG module is printed as:
`<status>--<flags> <name><opts> <type> <if-features>`
- `<status>` is one of:
 - + for current
 - x for deprecated
 - o for obsolete
- `<flags>` is one of:
 - rw for configuration data nodes and choice nodes
 - ro for non-configuration data nodes and choice nodes, output parameters to rpcs and actions, and notification parameters
 - w for input parameters to rpcs and actions
 - u for uses of a grouping
 - x for rpcs and actions
 - n for notifications
 - mp for nodes containing a "mount-point" extension statement

YANG Tree Diagram Syntax (2/3)

- Each node in a YANG module is printed as:
`<status>--<flags> <name><opts> <type> <if-features>`

- `<name>` is the name of the node
(`<name>`) means that the node is a choice node
:(`<name>`) means that the node is a case node

If the node is augmented into the tree from another module, its name is printed as `<prefix>:<name>`, where `<prefix>` is the prefix defined in the module where the node is defined.

If the node is a case node, there is no space before the `<name>`.

- `<opts>` is one of:
 - ? for an optional leaf, choice, anydata, or anyxml
 - ! for a presence container
 - * for a leaf-list or list
 - [`<keys>`] for a list's keys
 - / for a top-level data node in a mounted module
 - @ for a top-level data node of a module identified in a mount point parent reference

YANG Tree Diagram Syntax (3/3)

- Each node in a YANG module is printed as:

`<status>-<flags> <name><opts> <type> <if-features>`

- `<type>` is the name of the type for leafs and leaf-lists

If the type is a leafref, the type is printed as either

- 1) `"-> TARGET"`, where TARGET is the leafref path, with prefixes removed if possible or
- 2) `"leafref"`.

- `<if-features>` is the list of features this node depends on, printed within curly brackets and a question mark `"{...}?"`